



Università degli Studi di Padova

Laurea: Informatica

Corso: Ingegneria del Software

Anno Accademico: 2025/2026



Gruppo RubberDuck

email: GroupRubberDuck@gmail.com

Specifica Tecnica

Stato	In lavorazione
Versione	0.8.0
Autori	Aldo Bettega Davide Lorenzon Ana Maria Draghici Filippo Guerra Felician Mario Necsulescu Davide Testolin
Verificatori	Davide Lorenzon Filippo Guerra Felician Mario Necsulescu
Uso	Esterno
Destinatari	Prof. Tullio Vardanega Prof. Riccardo Cardin BlueWind srl

Vers.	Data	Autore	Verificatore	Descrizione
0.8.0	2026-04-22	Ana Maria Draghici		Stesura e scomposizione sezioni a partire dalla <u>Sezione 5.2</u> : Device, Asset e Session
0.7.3	2026-04-22	Davide Testolin	Ana Maria Draghici	Migliorata sezione <u>Sezione 3.6</u>
0.7.2	2026-04-20	Felician Mario Necsulescu	Davide Lorenzon	Stesura dei design pattern comportamentali e architeturali <u>Sezione 4.5</u> , <u>Sezione 4.2</u>
0.7.1	2026-04-19	Felician Mario Necsulescu	Davide Lorenzon	Stesura dei design pattern: creazionali e strutturali <u>Sezione 4.3</u> , <u>Sezione 4.4</u>
0.7.0	2026-04-19	Felician Mario Necsulescu	Davide Lorenzon	Stesura dei principi di design <u>Sezione 4.1</u>
0.6.4	2026-04-19	Davide Lorenzon	-	Revisione architetture della sezione relativa alla modifica degli asset <u>Sezione 5.3</u>
0.6.3	2026-04-19	Davide Lorenzon	-	Bozza delle classi relative alla generazione del report di conformità <u>Sezione 5.7</u>
0.6.2	2026-04-19	Davide Lorenzon	-	Bozza delle classi relative a import ed export tramite file di dispositivo e modello <u>Sezione 5.5</u> e <u>Sezione 5.6</u>
0.6.1	2026-04-17	Filippo Guerra	-	Bozza della classe valutazione <u>Sezione 5.4</u>

Vers.	Data	Autore	Verificatore	Descrizione
0.6.0	2026-04-17	Filippo Guerra	Davide Lorenzon	Stesura vista_dati Sezione 5.3
0.5.0	2026-04-16	Ana Maria Draghici	Davide Lorenzon	Stesura vista_dati Sezione 3.6
0.4.0	2026-04-16	Aldo Bettega	Davide Lorenzon	Stesura Sezione 5.1 e Sezione 5.2
0.3.0	2026-04-15	Aldo Bettega	Davide Lorenzon	Stesura della Sezio- ne 3.4
0.2.0	2026-04-10	Aldo Bettega	Davide Lorenzon	Stesura della Sezio- ne 3.1 e Sezio- ne 3.2
0.1.2	2026-04-09	Davide Lorenzon	Filippo Guerra	Bozza iniziale della Sezione 3.3 Archi- tettura di deploy- ment.
0.1.1	2026-04-08	Davide Lorenzon	Aldo Bettega	Bozza iniziale della Sezione 2 Tecnolo- gie.
0.1.0	2026-04-08	Aldo Bettega	Felician Mario Necsulescu	Stesura introduzio- ne
0.0.1	2026-04-08	Davide Lorenzon	Aldo Bettega	Creazione del docu- mento

Indice

1) Introduzione	1
1.1) Scopo del documento	1
1.2) Scopo del prodotto	1
1.3) Glossario	1
1.4) Riferimenti	2
1.4.1) Riferimenti normativi	2
1.4.2) Riferimenti informativi	2
2) Tecnologie	3
2.1) Backend	3
2.2) Frontend	5
2.3) Persistenza dei dati	6
3) Architettura di Sistema	7
3.1) Premessa: approccio di lavoro usato	7
3.2) Architettura Logica	7
3.2.1) Stile architetturale	7
3.2.2) Motivazioni della scelta architetturale	8
3.2.3) Limiti dell'architettura	8
3.2.4) Diagramma dei package	9
3.3) Architettura di Deployment	10
3.3.1) Motivazioni della scelta	10
3.3.2) Realizzazione Pratica e Topologia	10
3.4) Diagrammi di sequenza	11
3.4.1) UC05 - Importazione dispositivo	11
3.4.2) UC26, UC27 - Valutazione di un nodo e transizione di stato	12
3.4.3) UC30 - Esportazione report di conformità	13
3.5) Diagrammi di attività	14
3.5.1) Navigazione degli alberi	14
3.6) Schema dati	15
3.6.1) Scelte Architetturali e Formalismo	15
3.6.2) Schema Dati Device	16
3.6.3) Schema ComplianceStandard	18
3.6.4) Gestione degli Esiti e Storicità	21
4) Design Patterns	22
4.1) Principi di Design	22
4.1.1) Dependency Injection	22
4.2) Pattern Architetturali	22
4.2.1) Repository	22
4.2.2) Unit of Work	23
4.3) Design Patterns Creazionali	23
4.3.1) Factory	23

4.4) Design Patterns strutturali	24
4.4.1) Adapter	24
4.4.2) Facade	24
4.5) Design Patterns Comportamentali	25
4.5.1) Strategy	25
4.5.2) Template Method	25
5) Diagrammi delle classi	26
5.1) Diagramma di dominio	26
5.2) Device	28
5.2.1) WriteDeviceModule	28
5.2.2) CreateDevice	28
5.2.2.1) WriteDeviceController	29
5.2.2.2) CreateDeviceUseCase	29
5.2.2.3) CreateDeviceCommand	30
5.2.2.4) CreateDeviceService	30
5.2.2.5) Device	31
5.2.2.6) RegisterDevicePort	32
5.2.2.7) MongoDeviceAdapter	32
5.2.3) DeleteDevice	33
5.2.3.1) DeleteDeviceUseCase	34
5.2.3.2) DeleteDeviceService	34
5.2.3.3) DeleteDevicePort	35
5.2.4) SaveDevice	36
5.2.4.1) SaveDeviceUseCase	37
5.2.4.2) SaveDeviceCommand	37
5.2.4.3) SaveDeviceService	38
5.2.4.4) SaveDevicePort	38
5.2.5) ReadDeviceModule	39
5.2.6) GetDeviceDetail	40
5.2.6.1) QueryDeviceController	40
5.2.6.2) GetDeviceDetailUseCase	41
5.2.6.3) GetDeviceDetailService	41
5.2.6.4) FindDevicePort	42
5.2.7) GetDeviceList	43
5.2.7.1) GetDeviceListUseCase	44
5.2.7.2) GetDeviceListService	44
5.2.7.3) DeviceSummary	45
5.2.7.4) FindAllDevicesPort	45
5.3) Asset	46
5.3.1) CreateAsset	46
5.3.1.1) WriteAssetController	47

5.3.1.2)	CreateAssetUseCase	48
5.3.1.3)	CreateAssetCommand	48
5.3.1.4)	CreateAssetService	49
5.3.1.5)	Asset	49
5.3.1.6)	EvaluationSession	50
5.3.1.7)	InMemoryEvaluationSessionCache	50
5.3.1.8)	SaveSessionPort	51
5.3.1.9)	GetSessionPort	52
5.3.2)	SaveAsset	53
5.3.2.1)	SaveAssetUseCase	54
5.3.2.2)	SaveAssetCommand	54
5.3.2.3)	SaveAssetService	55
5.3.3)	DeleteAsset	56
5.3.3.1)	DeleteAssetUseCase	57
5.3.3.2)	DeleteAssetService	57
5.3.4)	GetAssetDetail	58
5.3.4.1)	QueryDashboardController	59
5.3.4.2)	GetAssetDetailUseCase	60
5.3.4.3)	GetAssetDetailService	60
5.3.4.4)	AssetDetail	61
5.3.4.5)	RequirementEval	62
5.3.4.6)	EvaluationSheet	62
5.4)	Session	63
5.4.1)	OpenEvaluationSession	63
5.4.1.0.1)	EvaluationSessionController	64
5.4.1.0.2)	OpenEvaluationSessionUseCase	65
5.4.1.0.3)	OpenEvaluationSessionService	65
5.4.1.0.4)	SessionCoordinator	66
5.4.1.0.5)	CreateSessionPort	66
5.4.1.0.6)	FindStandardPort	67
5.4.1.0.7)	MongoStandardAdapter	67
5.4.1.1)	SaveEvaluationSession	68
5.4.1.1.1)	SaveEvaluationSessionUseCase	69
5.4.1.1.2)	SaveEvaluationSessionService	69
5.4.1.1.3)	<i>SaveSessionPort</i>	70
5.4.1.2)	CloseEvaluationSession	71
5.4.1.2.1)	CloseEvaluationSessionUseCase	72
5.4.1.2.2)	CloseEvaluationSessionService	72
5.4.1.2.3)	DeleteSessionPort	73
5.4.1.3)	CommitEvaluationSession	74
5.4.1.3.1)	CommitEvaluationSessionService	75

5.4.1.3.2)	CommitEvaluationSessionUseCase	75
5.4.1.4)	CommitCloseSession	76
5.4.1.4.1)	CommitCloseEvaluationSessionService	77
5.5)	Importazione ed Esportazione Dispositivi	77
5.5.1)	Importazione Dispositivo	78
5.5.2)	Esportazione dispositivo	79
5.5.3)	Classi Condivise	80
5.6)	Importazione ed Esportazione Modelli	80
5.6.1)	Importazione Modello	80
5.6.2)	Esportazione modello	81
5.6.3)	Classi Condivise	82
5.7)	Generazione Report	82
6)	Tracciamento	84
7)	Qualità architetturale	85
8)	Gestione Errori e Logging	86
9)	Sicurezza	87
10)	Performance e Scalabilità	88

Lista delle tabelle

Tabella 1	Backend	3
Tabella 2	Frontend	5
Tabella 3	Schema dati: Root — Device	17
Tabella 4	Schema dati: Sub-documento — evaluation_cs	17
Tabella 5	Schema dati: Sub-documento — assets[N]	17
Tabella 6	Schema dati: Sub-documento — evaluations[N]	18
Tabella 7	Schema dati: Root — ComplianceStandard	20
Tabella 8	Schema dati: Sub-documento — requirements[N]	20
Tabella 9	Schema dati: Sub-documento — DecisionNode	20
Tabella 10	Schema dati: Sub-documento — LeafNode	21

Lista delle immagini

Figura 1	Schema logico: Documento Device	16
Figura 2	Schema logico: ComplianceStandard	19
Figura 3	Modulo di scrittura Dispositivi	28
Figura 4	Caso d'uso CreateDevice	28
Figura 5	WriteDeviceController	29
Figura 6	CreateDeviceUseCase	29
Figura 7	CreateDeviceCommand	30
Figura 8	CreateDeviceService	30
Figura 9	Device	31
Figura 10	RegisterDevicePort	32
Figura 11	MongoDeviceAdapter	32
Figura 12	Caso d'uso DeleteDevice	33
Figura 13	DeleteDeviceUseCase	34
Figura 14	DeleteDeviceService	34
Figura 15	DeleteDevicePort	35
Figura 16	Caso d'uso SaveDevice	36
Figura 17	SaveDeviceUseCase	37
Figura 18	SaveDeviceCommand	37
Figura 19	SaveDeviceService	38
Figura 20	SaveDevicePort	38
Figura 21	Modulo di lettura Dispositivi	39
Figura 22	GetDeviceDetail	40
Figura 23	QueryDeviceController	40
Figura 24	GetDeviceDetailUseCase	41
Figura 25	GetDeviceDetailService	41
Figura 26	FindDevicePort	42
Figura 27	Caso d'uso GetDeviceList	43
Figura 28	GetDeviceListUseCase	44
Figura 29	GetDeviceListService	44
Figura 30	DeviceSummary	45
Figura 31	FindAllDevicesPort	45
Figura 32	Modulo di scrittura Asset	46
Figura 33	Caso d'uso CreateAsset	46
Figura 34	WriteAssetController	47
Figura 35	CreateAssetUseCase	48
Figura 36	CreateAssetCommand	48
Figura 37	CreateAssetService	49
Figura 38	Asset	49
Figura 39	EvaluationSession	50
Figura 40	InMemoryEvaluationSessionCache	50

Figura 41	SaveSessionPort	51
Figura 42	GetSessionPort	52
Figura 43	Caso d'uso SaveAsset	53
Figura 44	SaveAssetUseCase	54
Figura 45	SaveAssetCommand	54
Figura 46	SaveAssetService	55
Figura 47	Caso d'uso DeleteAsset	56
Figura 48	DeleteAssetUseCase	57
Figura 49	DeleteAssetService	57
Figura 50	Diagramma delle classi – Caso d'uso GetAssetDetail	58
Figura 51	QueryDashboardController	59
Figura 52	GetAssetDetailUseCase	60
Figura 53	GetAssetDetailService	60
Figura 54	AssetDetail	61
Figura 55	RequirementEval	62
Figura 56	EvaluationSheet	62
Figura 57	Caso d'uso OpenEvaluationSession	63
Figura 58	EvaluationSessionController	64
Figura 59	OpenEvaluationSessionUseCase	65
Figura 60	OpenEvaluationSessionService	65
Figura 61	SaveAssetService	66
Figura 62	CreateSessionPort	66
Figura 63	FindStandardPort	67
Figura 64	MongoStandardAdapter	67
Figura 65	Caso d'uso SaveEvaluationSession	68
Figura 66	SaveEvaluationSessionUseCase	69
Figura 67	SaveEvaluationSessionService	69
Figura 68	SaveSessionPort	70
Figura 69	Caso d'uso CloseEvaluationSession	71
Figura 70	CloseEvaluationSessionUseCase	72
Figura 71	CloseEvaluationSessionService	72
Figura 72	DeleteSessionPort	73
Figura 73	Caso d'uso CommitEvaluationSession	74
Figura 74	CommitEvaluationSessionService	75
Figura 75	CommitEvaluationSessionUseCase	75
Figura 76	Caso d'uso CommitCloseSession	76
Figura 77	CommitCloseEvaluationSessionService	77
Figura 78	Diagramma delle classi - Importazione Dispositivo	78
Figura 79	Diagramma delle classi - Esportazione Dispositivo	79
Figura 80	Diagramma delle classi - Importazione Modello	80
Figura 81	Diagramma delle classi - Esportazione Modello	81

Figura 82	Diagramma delle classi - Importazione Modello	82
-----------	---	----

1) Introduzione

1.1) Scopo del documento

Il presente documento intende fornire una descrizione approfondita dell'architettura del prodotto software, delineando con chiarezza le componenti che lo costituiscono, le loro responsabilità e le interazioni reciproche all'interno del sistema. La Specifica Tecnica funge da riferimento principale per la fase di progettazione e codifica, garantendo coerenza con i requisiti analizzati e consolidando la maturità architetturale del prodotto.

Nello specifico, questo documento si propone di:

- **Definire l'architettura logica e i design pattern:** descrivere l'organizzazione dei moduli e le logiche di interazione, evidenziando come i pattern adottati garantiscano un codice modulare e testabile.
- **Motivare lo stack tecnologico:** giustificare la scelta delle tecnologie utilizzate in funzione della scalabilità del sistema e della qualità complessiva del prodotto finale.
- **Delineare l'architettura di deployment:** illustrare la distribuzione delle componenti negli ambienti di esecuzione e le strategie di rilascio adottate.
- **Fornire un framework per l'evoluzione del software :** stabilire le linee guida necessarie per facilitare la comprensione del sistema, agevolando futuri interventi di estensione e manutenzione.

1.2) Scopo del prodotto

Il prodotto si prefigge di automatizzare e digitalizzare il processo di verifica della conformità alla normativa EN 18031, sostituendo le attuali procedure manuali, spesso onerose e soggette a errore umano, con una soluzione software interattiva.

Il sistema è progettato per guidare l'utente attraverso l'esecuzione di alberi decisionali (Decision Tree) complessi, permettendo la valutazione sistematica dei requisiti di sicurezza per dispositivi IoT e la generazione di esiti certi (Pass, Fail, N.A.). L'integrazione di funzionalità per l'importazione di dati strutturati e una dashboard di monitoraggio in tempo reale mira a garantire la massima tracciabilità del processo, ottimizzando i tempi operativi della proponente e assicurando un elevato standard di affidabilità e manutenibilità dei risultati ottenuti.

1.3) Glossario

Per garantire precisione terminologica senza appesantire la lettura, in questo documento i termini tecnici presenti nel Glossario sono segnalati con un pedice "G", ad esempio:

termine_G: indica che il termine è definito nel Glossario e può essere consultato per chiarimenti.

Questo metodo consente di mantenere il testo chiaro e tecnicamente corretto, permettendo al lettore di riferirsi al Glossario solo quando necessario, senza interrompere il flusso della lettura.

1.4) Riferimenti

1.4.1) Riferimenti normativi

- [Capitolato d'appalto C1 - Automated EN18031 Compliance Verification di BlueWind Srl](#)
Ultima consultazione: 8 aprile 2026;
- [Regolamento del progetto](#)
Ultima consultazione: 8 aprile 2026;
- [Standard ISO/IEC/IEEE 12207:2017](#)
Ultima consultazione: 10 gennaio 2026;
- [Standard ISO/IEC 9126](#)
Ultima consultazione: 10 gennaio 2026;

1.4.2) Riferimenti informativi

- [Glossario del gruppo](#)
Ultima consultazione: 8 aprile 2026;
- [Software Engineering, Ian Sommerville](#)
Ultima consultazione: 10 gennaio 2026;
- [Documentazione del gruppo GroupRubberDuck](#)
Ultima consultazione: 8 aprile 2026;

2) Tecnologie

In questa sezione vengono descritte le tecnologie utilizzate nello sviluppo del progetto **Automated EN18031 Compliance Verification**.

Per ogni tecnologia è riportata la versione di riferimento e il relativo ruolo all'interno del progetto

2.1) Backend

Nome	Versione	Descrizione
Linguaggio di Programmazione		
Python	3.12.X	Linguaggio interpretato scelto per la rapidità di sviluppo, l'alta leggibilità e il vasto ecosistema.
Framework Principale		
Flask	3.1.3	Micro-framework web utilizzato come infrastruttura principale di backend. Consente una gestione flessibile e leggera del routing per esporre le interfacce API. Buona documentazione e possibilità di confronto tecnico con la proponente.
Dipendenze principali		
waitress	3.0.2	Server WSGI leggero. Gestisce la concorrenza delle richieste in modo affidabile.
fpdf		Libreria per la generazione dinamica di documenti e reportistica in formato PDF direttamente lato server.
pymongo		Driver ufficiale per l'integrazione con MongoDB.
pydantic		Utilizzato per la validazione rigida e type-safe dei dati in ingresso e delle variabili d'ambiente di sistema.
python-dotenv		Gestione semplificata delle configurazioni e delle variabili d'ambiente.
Jinja2		Template engine HTML con un'ottima integrazione con Flask.

Nome	Versione	Descrizione
werkzeug		Server di sviluppo locale.
watchdog		Permette l'hot reloading, passando le modifiche al server di sviluppo senza necessità di riavvio e preservando lo stato dell'applicazione.
Dynamic Testing		
pytest		Framework di testing adottato per la sua sintassi concisa. Utilizzato per automatizzare i «Sanity Tests» e validare l'integrazione di sistema alla radice del progetto.
Static Testing		
ruff		Lint e formatter estremamente veloce (scritto in Rust). Impone e garantisce standard di codice puliti e uniformi senza rallentare lo sviluppo.
mypy		Analizzatore statico basato sul Type Hinting. Previene i bug e gli errori di tipo a tempo di sviluppo prima ancora dell'esecuzione
Sviluppo		
poetry		Gestore moderno delle dipendenze e degli ambienti virtuali (.venv). Garantisce build riproducibili tra i vari sviluppatori tramite il file di blocco (poetry.lock).

Tabella 1: Backend

2.2) Frontend

Nome	Versione	Descrizione
Linguaggio di Programmazione		
JavaScript		Linguaggio client-side universale.
Framework Principale		
Vue.js		Framework progressivo con una curva di apprendimento morbida e un'ottima implementazione della reattività. L'adozione della Composition API permette di scrivere logica modulare e riutilizzabile.
Dipendenze principali		
pinia		Gestore dello stato ufficiale e leggero per Vue.
axios		Client HTTP flessibile. Preferito alla Fetch API nativa per l'ergonomia nella gestione automatica del JSON, la gestione degli errori e l'uso degli interceptor per le chiamate verso le API di Flask.
tailwindcss		Framework CSS utility-first per un'agile stilizzazione dell'interfaccia.
Dynamic Testing		
vitest		Framework di unit testing veloce e nativo per Vite. Utilizzato per validare la logica isolata dei componenti.
vue-test-utils		Libreria ufficiale affiancata a Vitest per montare e simulare le interazioni dell'utente sulle singole isole Vue durante i test.
Static Testing		
eslint		Linter per garantire standard di pulizia e coerenza nel codice JavaScript dei vari componenti Vue.
Sviluppo		

Nome	Versione	Descrizione
vite		Tool di build per minificare e raggruppare i file JS/CSS corrispondenti ai componenti Vue, ottimizzandoli per la produzione.

Tabella 2: Frontend

2.3) Persistenza dei dati

MongoDB è un database NoSQL orientato ai documenti. A differenza dei classici database relazionali (che usano tabelle e colonne rigide), archivia le informazioni in documenti flessibili molto simili al formato JSON.

La versione utilizzata è la 7.XX

Permette di aggiungere o rimuovere campi dai dati senza riscrivere l'intera organizzazione delle informazioni del database.

Nel progetto **Automated EN18031 Compliance Verification** viene utilizzato come sistema per la persistenza dei dati, in particolare per la rappresentazione dei dispositivi sottoposti alle valutazioni e per la rappresentazione del modello di standard tramite configurazioni esterne.

Il suo utilizzo permette di rappresentare facilmente strutture dati annidate, tramite una sintassi JSON.

Evita la gestione di join complessi tipici di un database relazionale, offre una maggiore sicurezza rispetto alla gestione manuale di file di rappresentazione interni.

È facilmente integrabile nel sistema backend

3) Architettura di Sistema

3.1) Premessa: approccio di lavoro usato

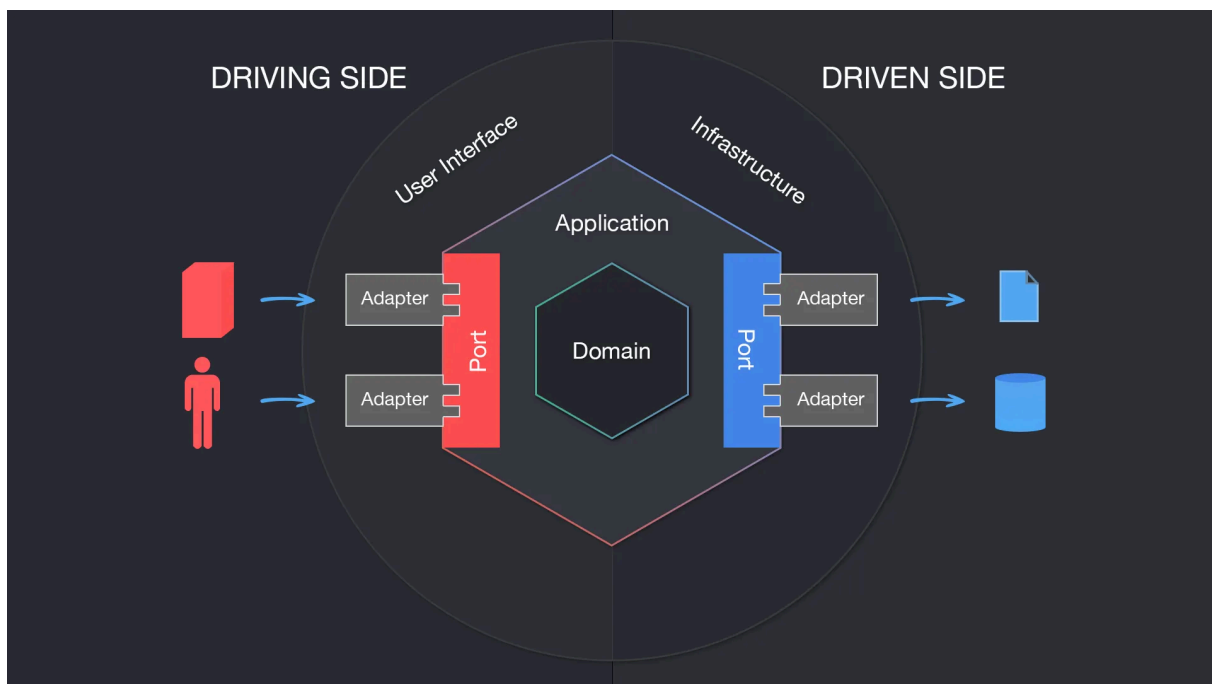
L'obiettivo della fase di progettazione è definire un'architettura software in grado di soddisfare pienamente i requisiti di sistema, operando secondo criteri di sostenibilità e ottimizzazione delle risorse (design-to-cost).

La metodologia adottata per la definizione dell'architettura è di tipo Top-Down. Il sistema viene inizialmente concepito nella sua interezza come un'unica entità astratta e, attraverso un processo di esplorazione funzionale, viene progressivamente decomposto in sotto-sistemi logici più piccoli e gestibili.

3.2) Architettura Logica

3.2.1) Stile architetturale

L'architettura adottata per realizzare il prodotto è l'architettura esagonale. Questo tipo di architettura ha come obiettivo primario isolare la logica di business dal resto. Questo approccio garantisce un'elevata testabilità del sistema e rende il nucleo del software completamente agnostico rispetto ai dettagli implementativi (framework, database o interfacce). Il sistema è strutturato in livelli concentrici, organizzati come segue:



- **Domain (Core):** Rappresenta il nucleo centrale dell'applicazione. Contiene la logica di business pura ed è rigorosamente privo di dipendenze esterne.
- **Ports:** Costituiscono il punto di connessione tra il nucleo e il mondo esterno, permettendo una comunicazione strutturata senza creare accoppiamento. Si suddividono in Inbound Ports (definiscono i casi d'uso accessibili dall'esterno) e Outbound Ports (permettono al nucleo di definire interfacce per interagire con i servizi esterni).

- **Adapters:** Rappresentano lo strato più esterno e fungono da traduttori tra le tecnologie specifiche e il nucleo. Si suddividono in Driving/Input Adapters (adattatori in entrata, che guidano l'applicazione ricevendo input e invocando le Inbound Ports) e Driven/Output Adapters (adattatori in uscita, che vengono guidati dall'applicazione per interagire con l'infrastruttura esterna tramite le Outbound Ports).

3.2.2) Motivazioni della scelta architetturale

Le motivazioni per le quali questa architettura è stata scelta sono le seguenti:

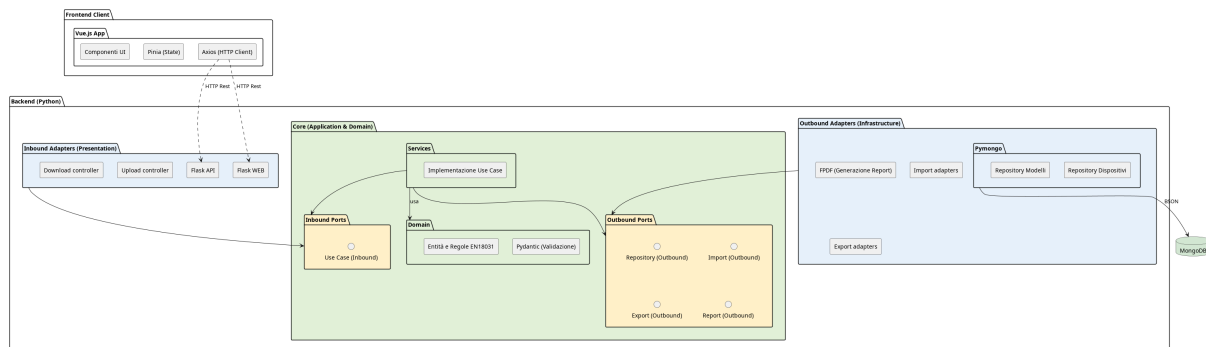
- **Testabilità:** l'assenza di dipendenze esterne nel livello di Dominio permette di scrivere Unit Test in puro Python in modo semplice e veloce. È possibile testare la logica di business in modo isolato, iniettando mock o stub per simulare le interfacce, senza la necessità di avviare il server Flask o connettersi al database MongoDB.
- **Divisione del nucleo:** il nucleo dell'applicazione è totalmente disaccoppiato dall'infrastruttura. Essendo ignaro del livello di presentazione (l'interfaccia utente in HTML, CSS e Vue.js) e del livello di persistenza (il database NoSQL MongoDB), il sistema garantisce un'alta tolleranza ai cambiamenti. È possibile sostituire o aggiornare le tecnologie esterne senza dover alterare la logica di business.
- **Sviluppo parallelo:** la rigorosa definizione dei contratti di comunicazione (le Porte) nelle fasi iniziali di progettazione permette al team di procedere in parallelo. Frontend, backend e integrazione con il database possono essere sviluppati simultaneamente da membri diversi del gruppo, riducendo i colli di bottiglia.
- **Inversione delle dipendenze:** l'architettura applica rigorosamente questo principio: è il Dominio a dettare i contratti (le interfacce) a cui i servizi esterni devono sottostare, e non viceversa. In questo modo, il nucleo dell'applicazione mantiene il controllo assoluto del flusso ed è protetto dai potenziali effetti collaterali (side effects) derivanti dall'infrastruttura.

3.2.3) Limiti dell'architettura

Gli aspetti negativi di questa scelta sono:

- **Ripida curva di apprendimento:** l'architettura esagonale richiede una profonda comprensione dei principi SOLID, in particolare la Dependency Injection e usare questo pattern per la prima volta richiede profondo studio. Questo rischio sarà mitigato da una precisa fase di progettazione che semplificherà la codifica.
- **Elevato overhead iniziale:** La ferrea separazione dei livelli impone la stesura di un'abbondante quantità di codice infrastrutturale (boilerplate). Risulta necessario definire contratti astratti (Porte), implementazioni concrete (Adattatori) e orchestratori (Servizi), allungando i tempi di sviluppo nelle prime fasi del progetto. Il gruppo ha tuttavia accettato questo costo iniziale, ritenendolo un investimento necessario a fronte del drastico abbattimento dei futuri costi di manutenzione e della massima testabilità garantita al nucleo applicativo.

3.2.4) Diagramma dei package



Il diagramma illustra l'organizzazione logica del sistema Automated EN18031 Compliance Verification, fondata sull'architettura esagonale. Il sistema è strutturalmente ripartito in un livello di presentazione esterno (Frontend Client), sviluppato tramite il framework Vue.js, e un nucleo applicativo (Backend), implementato in Python.

Per garantire una rigorosa separazione delle responsabilità e il pieno rispetto del principio di Inversione delle Dipendenze, il Backend si articola nei seguenti livelli tipici dell'architettura esagonale:

1. **Adapters:** Costituisce il confine tecnologico del sistema, responsabile della comunicazione con l'esterno. Si divide in:
 - **Inbound Adapters** (Driving Adapters): Agiscono da punto di ingresso per il sistema. Intercettano gli input esterni (es. le richieste HTTP/REST inviate dal Frontend), ne effettuano il parsing e traducono tali direttive in invocazioni dirette verso le Inbound Ports, senza mai accedere alla logica di business.
 - **Outbound Adapters** (Driven Adapters): Rappresentano le tecnologie concrete di uscita (es. il driver MongoDB o le librerie di generazione PDF). Il loro compito è implementare materialmente le interfacce definite nel livello Outbound Ports, traducendo i comandi astratti in operazioni specifiche per l'infrastruttura.
2. **Core** (Nucleo Applicativo): È il cuore del software, totalmente agnostico e privo di dipendenze verso framework web o database. Al suo interno è stratificato in:
 - **Ports:** Funge da barriera di isolamento. Definisce le interfacce in puro Python, disaccoppiando la tecnologia dall'applicazione:
 - **Inbound Ports** (Primary Ports): Definiscono le interfacce (i contratti) relative ai Casi d'Uso (Use Cases) che il sistema offre. Specificano «cosa» il sistema è in grado di fare, permettendo agli Inbound Adapters di invocare le operazioni senza dover conoscere l'implementazione sottostante.
 - **Outbound Ports** (Secondary Ports): Definiscono i contratti astratti di cui il dominio ha bisogno per comunicare con l'esterno (ad esempio, le interfacce del Repository Pattern per l'accesso ai dati). Queste porte vengono usate dal livello Services e implementate dagli Outbound Adapters.

- **Services:** Costituiscono il livello di orchestrazione. Implementano concretamente i Casi d'Uso definiti nelle Inbound Ports, coordinano il flusso di esecuzione richiamando le entità di dominio, e si avvalgono delle Outbound Ports per la persistenza dei dati.
- **Domain:** Incapsula le regole di business pure e la logica della norma EN18031. Contiene la modellazione formale delle entità e la validazione strutturale dei dati (supportata da Pydantic), operando in totale e completo isolamento.

3.3) Architettura di Deployment

Il sistema adotta un'architettura a Monolite Modulare.

L'intera applicazione è concepita, sviluppata e rilasciata come un'unica unità eseguibile.

3.3.1) Motivazioni della scelta

Questa scelta è coerente con lo sviluppo di una web app locale.

In questo contesto un'architettura a monolite modulare, rispetto all'architettura a microservizi, offre i seguenti vantaggi:

- **Semplicità di Deployment:** tutte le funzionalità sono contenute nella singola unità operativa;
- **Latenze ridotte:** lo scambio di informazioni avviene completamente in locale.

Non si è optato per una architettura monolitica tradizionale in cui spesso il codice è fortemente accoppiato.

La rigorosa divisione in moduli logici isolati permette di coniugare la semplicità di rilascio dell'applicazione come singola unità con alcuni dei vantaggi organizzativi tipici delle architetture a microservizi:

- **Separazione delle responsabilità** tra i vari ambiti del dominio.
- **Semplicità di sviluppo e manutenibilità** a lungo termine.
- **Facilità di testing**, permettendo di collaudare i singoli moduli in totale isolamento.

3.3.2) Realizzazione Pratica e Topologia

Per la distribuzione viene usato Docker come tool di containerizzazione e Docker Compose come tool di orchestrazione.

1. Nodi di Esecuzione:

- **Browser dell'utente**, l'interfaccia utente è realizzata tramite browser;
- **Server Backend**, contiene la core logic dell'applicazione e risponde alle richieste dell'utente;
- **Database MongoDB**, sistema di permanenza;

2. Packaging e isolamento. L'infrastruttura è orchestrata tramite Docker Compose e prevede i seguenti container:

- **Web-app**

Contiene l'ambiente Python, il framework Flask, gli asset statici del frontend e altre dipendenze¹

- **Mongodb**

Basato sull'immagine ufficiale di MongoDB. Per prevenire la perdita dei dati questo container è agganciato a un Docker Volume locale.

3. Comunicazioni e protocolli di rete

I nodi comunicano attraverso i seguenti canali e protocolli:

- **Tra Browser e Backend:**

La comunicazione avviene in chiaro tramite protocollo HTTP/REST

- Tra Backend e Database: Flask comunica con MongoDB utilizzando il protocollo binario proprietario.

La porta del database non è esposta verso il browser né verso l'esterno, garantendo l'integrità dei dati.

4. Gestione del Traffico Web

Tra il browser dell'utente e l'applicazione Python è interposto un server **WSGI**.

Questo strato intermedio garantisce stabilità, gestisce la concorrenza delle richieste e le instrada in modo sicuro verso la logica applicativa.

3.4) Diagrammi di sequenza

La seguente sezione illustra il comportamento dinamico del sistema tramite diagrammi di sequenza, focalizzandosi sui casi d'uso di maggiore interesse. Questi modelli descrivono l'ordine cronologico dei messaggi scambiati tra gli attori esterni, i componenti infrastrutturali e il nucleo applicativo.

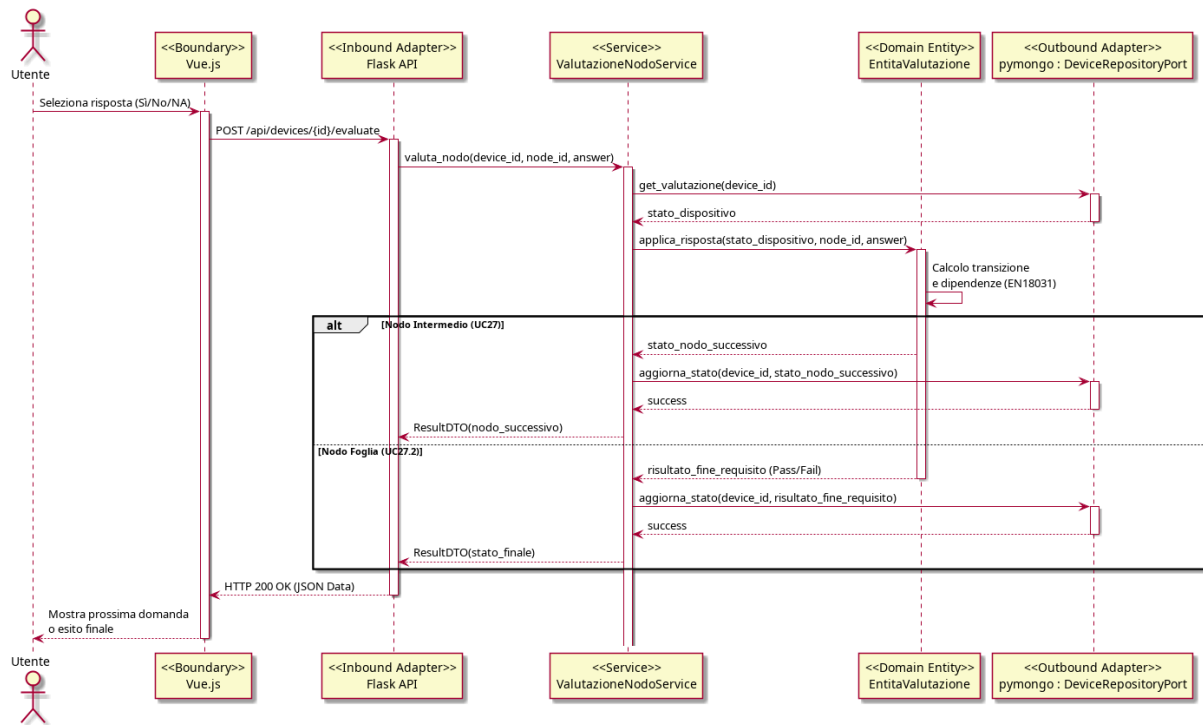
3.4.1) UC05 - Importazione dispositivo

Il diagramma illustra il processo di importazione e validazione strutturale di un dispositivo attraverso i layer dell'Architettura Esagonale. Il flusso adotta un approccio Fail-Fast diviso in due fasi. Inizialmente, l'Adattatore Inbound utilizza un Data Transfer Object (DTO) per eseguire una validazione strutturale sul formato del file in ingresso. Successivamente, il Servizio estrae i dati validati e li passa al Dominio, a cui è delegata esclusivamente la verifica delle regole normative.

Tramite le due aree alt viene modellata la gestione degli errori del caso (UC06): il primo blocco respinge i payload malformati fermandoli al confine del sistema (HTTP 400); il secondo gestisce le violazioni delle regole di business sollevate dal nucleo applicativo (HTTP 422). Solo se l'entità supera entrambi i controlli, il Servizio invoca l'Adattatore di persistenza per il salvataggio e completa l'operazione.

¹Vedi sezione [Tecnologie - Backend](#)

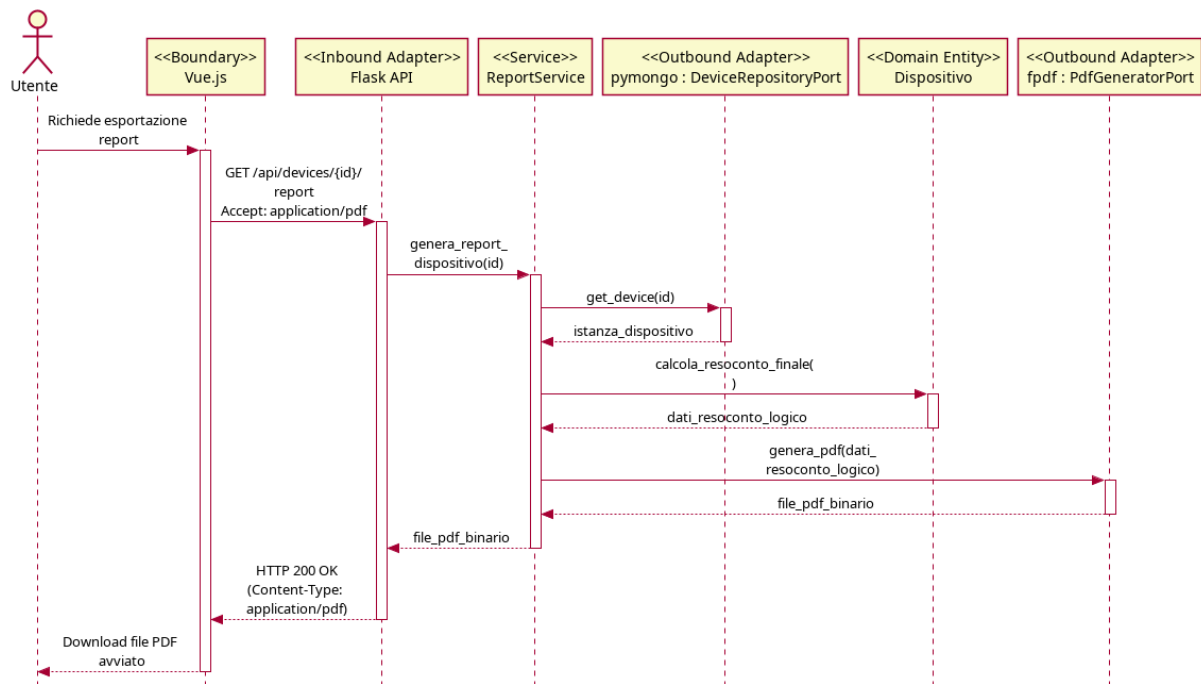
3.4.2) UC26, UC27 - Valutazione di un nodo e transizione di stato



Il diagramma di sequenza illustra il flusso principale di interazione durante la valutazione di un dispositivo secondo la norma EN18031. Il diagramma evidenzia il rigoroso attraversamento dei layer architetturici, dal Boundary (Vue.js) fino al driver di persistenza (PyMongo).

Degno di nota è l'isolamento del livello Domain: il frontend e gli adapter non conoscono le regole normative. È il Servizio applicativo che recupera lo stato, lo inietta nel Dominio, e si affida al calcolo di quest'ultimo. Tramite l'uso di una sezione alt, il diagramma modella il comportamento polimorfico del flusso: se la risposta porta a un nodo intermedio, il sistema salva il progresso e restituisce la domanda successiva; se la risposta raggiunge una foglia dell'albero normativo, il sistema calcola la conformità finale (Pass/Fail) e chiude lo stato della valutazione.

3.4.3) UC30 - Esportazione report di conformità



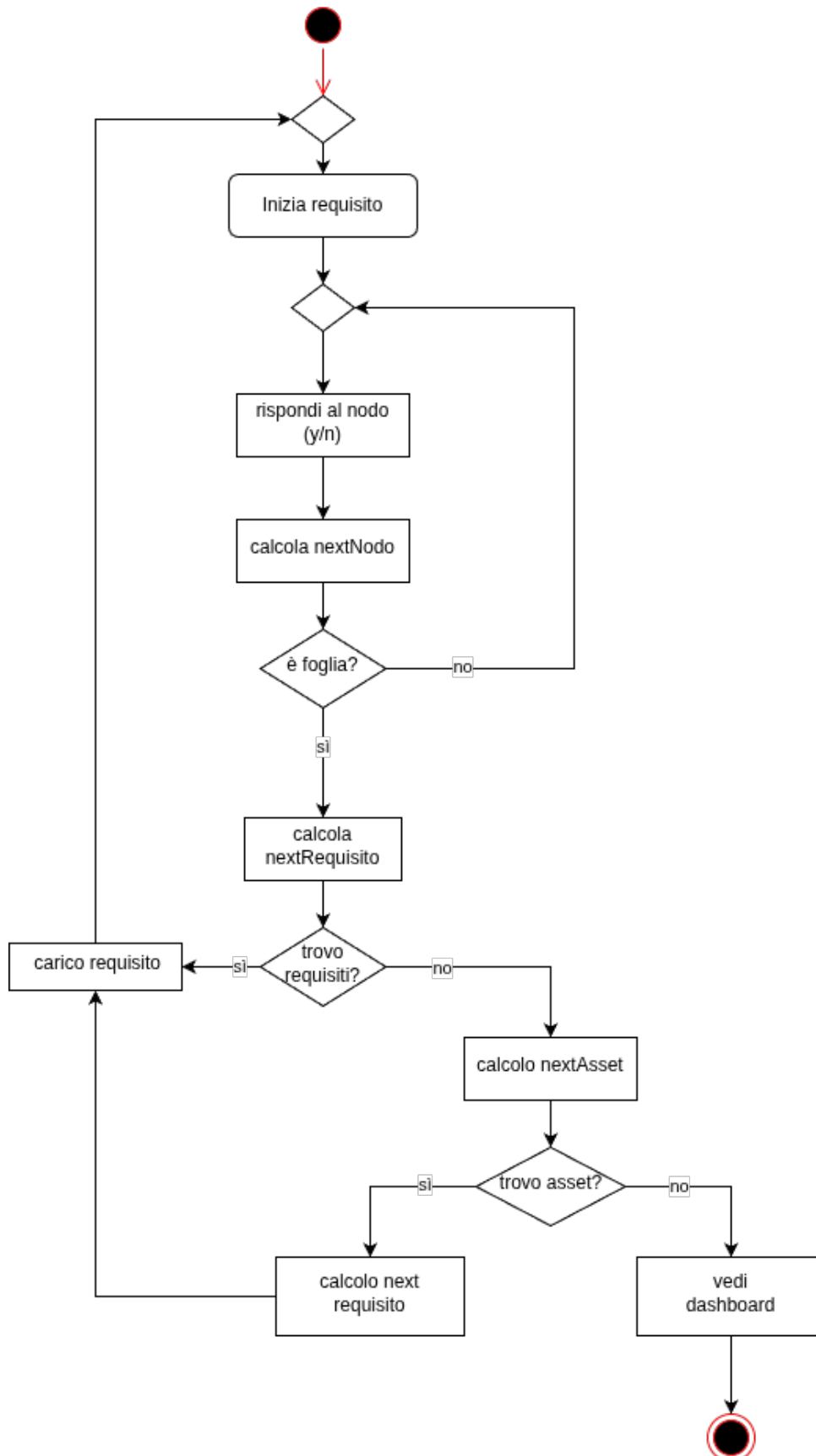
Il diagramma di sequenza illustra il processo di generazione ed esportazione del resoconto finale di conformità per un dispositivo valutato.

Il flusso è innescato da una chiamata HTTP gestita dall'Adattatore Inbound. Il Servizio applicativo avvia il recupero del documento che viene effettuato grazie all'outbound adapter (PyMongo) che estrae i dati dal database.

Una volta recuperato il dispositivo, il Servizio invoca l'aggregazione dei verdeti sul Dominio, il quale restituisce una struttura dati esclusivamente logica (Pass/Fail/NA) senza possedere alcuna conoscenza del rendering finale. Per la creazione del file fisico il Servizio usa la porta di generazione del pdf che traduce i dati puri in un layout grafico, restituendo il file che arriverà all'utente.

3.5) Diagrammi di attività

3.5.1) Navigazione degli alberi



Il diagramma di attività illustra l'algoritmo di navigazione dell'albero normativo.

Il flusso si basa su un ciclo continuo il cui innesco principale è la risposta dell'utente a uno specifico nodo (Sì/No). Dopo l'inserimento dell'input, il sistema calcola il nodo successivo e ne verifica la natura tramite un blocco decisionale («è foglia?»):

Se il nodo non è una foglia (nodo intermedio), il flusso torna indietro per sottoporre all'utente la nuova domanda appena calcolata.

Se il nodo è una foglia (ramo di valutazione concluso), il sistema innesca la logica di avanzamento gerarchico.

La fase di avanzamento procede per livelli. Dapprima, il sistema calcola e verifica se vi sono ulteriori requisiti da valutare per l'asset corrente («trovo requisiti?»). In caso positivo, il nuovo requisito viene caricato e il ciclo di domande riparte dall'inizio. In caso negativo, il sistema sale di livello verificando l'esistenza di ulteriori asset non ancora esaminati nel dispositivo («trovo asset?»). Se viene individuato un nuovo asset, ne vengono calcolati i relativi requisiti, che vengono caricati per riavviare la compilazione.

L'algoritmo fuoriesce da questo ciclo annidato solo ed esclusivamente quando sia i requisiti sia gli asset del dispositivo sono stati completamente esauriti. In questo scenario conclusivo, il sistema torna alla pagina di dashboard.

3.6) Schema dati

In questa sezione viene illustrata l'organizzazione logica dei dati all'interno del database MongoDB.

3.6.1) Scelte Architetture e Formalismo

A differenza dei sistemi relazionali basati su tabelle, MongoDB è un database **NoSQL orientato ai documenti** con struttura *schema-flexible*. Questa scelta architeturale è ideale per gestire strutture dati gerarchiche e dinamiche (come gli alberi decisionali), evitando il sovraccarico computazionale derivante da query di JOIN complesse.

Per la modellazione visiva non si utilizzano diagrammi Entity-Relationship, in quanto non idonei ai database documentali.

Si adotta invece il formalismo di Hackolade (consultabile al link: https://hackolade.com/schemas/Yelp_Challenge_dataset_documentation.html), che descrive chiaramente la gerarchia dei campi, i tipi di dato e le nidificazioni.

Nota sulla validazione dei dati: Benché i documenti vengano illustrati con tipi logici (incluso il tipo `enum` per chiarezza espositiva), la validazione strutturale di base sarà garantita dall'adapter predisposto alle comunicazioni col database

Le due *collection* principali del database sono:

- **Collection «Compliance Standards»:** Catalogo di sola lettura dei template. Contiene le definizioni degli standard, le versioni e la logica degli alberi decisionali.
- **Collection «Devices»:** Registro dinamico dei device censiti. Memorizza lo stato delle valutazioni e il riferimento puntuale al modello normativo applicato.

3.6.2) Schema Dati Device

Il documento *Device* è l'entità centrale del sistema. La sua struttura gerarchica rispecchia la scomposizione fisica e funzionale dell'oggetto analizzato in tre blocchi logici:

1. **Header Identificativo (Root):** Informazioni anagrafiche, tecniche e puntatore al modello normativo.
2. **Lista Asset (Nodi Figli):** Array di sub-documenti che modellano le singole componenti o funzionalità del dispositivo.
3. **Registro Valutazioni (Nodi Terminali):** Dizionario delle risposte fornite per i requisiti applicabili a ciascun asset.

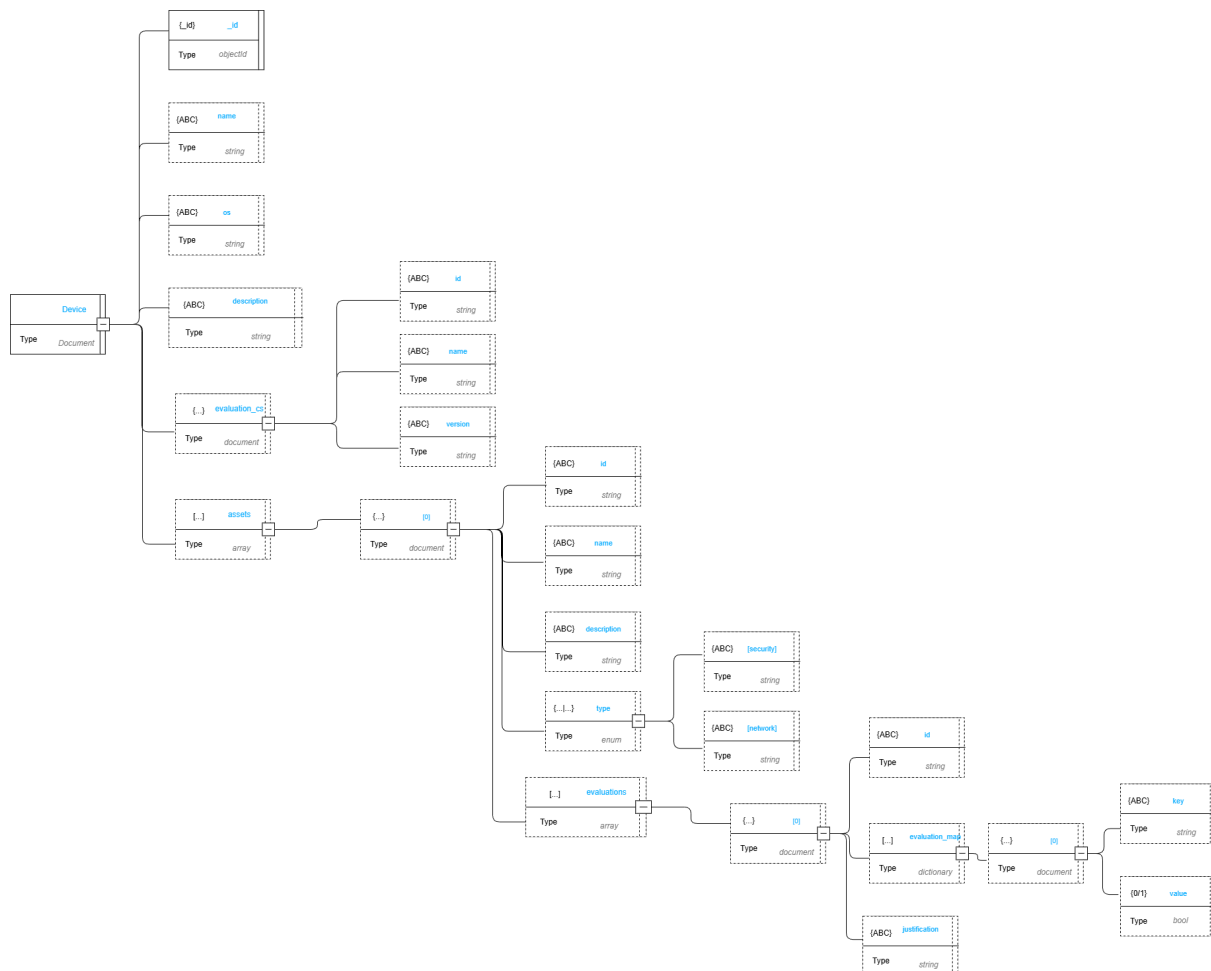


Figura 1: Schema logico: Documento Device

Root --- Device

Campo	Tipo BSON	Descrizione
<code>_id</code>	ObjectId	Identificativo univoco generato dal database.
<code>name</code>	string	Nome assegnato dall'utente al dispositivo. Valore obbligatorio.
<code>os</code>	string	Sistema operativo o firmware installato.
<code>description</code>	string	Testo libero descrittivo del dispositivo.
<code>evaluation_cs</code>	document	Metadati del modello di riferimento (compliance standard) usato per la valutazione (<code>id</code> , <code>name</code> , <code>version</code>).
<code>assets</code>	array	Elenco degli asset associati al dispositivo. Può essere inizializzato come array vuoto <code>[]</code> .

Tabella 3: Schema dati: Root — Device

Sub-documento --- evaluation_cs

Campo	Tipo BSON	Descrizione
<code>id</code>	string	Identificativo del modello di riferimento nel sistema esterno.
<code>name</code>	string	Nome leggibile del modello.
<code>version</code>	string	Versione del modello applicata alla valutazione.

Tabella 4: Schema dati: Sub-documento — evaluation_cs

Sub-documento --- assets[N]

Campo	Tipo BSON	Descrizione
<code>id</code>	string	ID applicativo per l'identificazione univoca dell'asset nel dispositivo.
<code>name</code>	string	Nome descrittivo dell'asset.
<code>description</code>	string	Testo libero descrittivo dell'asset.
<code>type</code>	enum	Categoria logica dell'asset. Valori ammessi: <code>security</code> , <code>network</code> .
<code>evaluations</code>	array	Elenco delle valutazioni fornite per questo asset. Può essere vuoto <code>[]</code> .

Tabella 5: Schema dati: Sub-documento — assets[N]

Sub-documento --- evaluations[N]

Campo	Tipo BSON	Descrizione
<code>id</code>	string	Riferimento al codice del requisito presente nel modello.
<code>evaluation_map</code>	dictionary	Mappa chiave-valore delle risposte. La chiave (string) corrisponde al codice del nodo decisionale, il valore (enum) rappresenta l'esito logico: <code>PASS</code> , <code>FAIL</code> , <code>NA</code> (Not Applicable).
<code>justification</code>	string	Testo descrittivo della motivazione. Obbligatorio a livello applicativo in caso di esito <code>FAIL</code> o <code>NA</code> .

Tabella 6: Schema dati: Sub-documento — evaluations[N]

3.6.3) Schema ComplianceStandard

La collection «Modelli» contiene la definizione strutturale degli standard normativi. A differenza del Dispositivo, orientato allo stato, il Modello descrive la logica di valutazione. Si suddivide in:

- **Metadati:** Versionamento e identificazione.
- **Albero dei Requisiti:** Struttura che definisce il testo della norma e la logica decisionale per guidare l'utente alla valutazione finale.

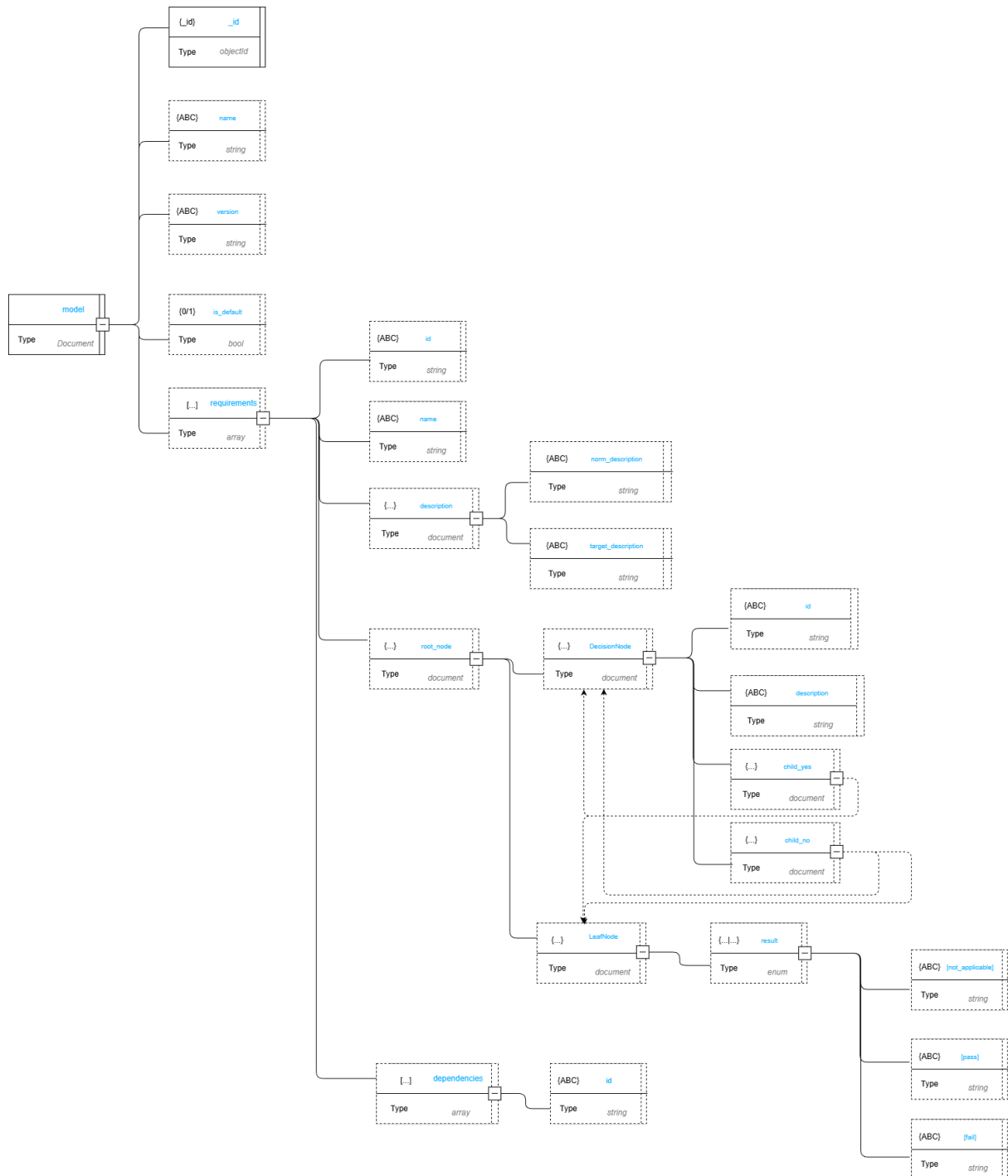


Figura 2: Schema logico: ComplianceStandard

Root --- ComplianceStandard

Campo	Tipo BSON	Descrizione
<code>_id</code>	ObjectId	Identificativo univoco del modello.
<code>name</code>	string	Nome dello standard normativo.
<code>version</code>	string	Identificativo della versione del modello.
<code>is_default</code>	bool	Flag che indica se il modello è quello predefinito per nuove valutazioni.
<code>requirements</code>	array	Elenco dei requisiti che compongono la normativa.

Tabella 7: Schema dati: Root — ComplianceStandard

Sub-documento --- requirements[N]

Campo	Tipo BSON	Descrizione
<code>id</code>	string	Identificativo testuale univoco del requisito.
<code>name</code>	string	Titolo sintetico del requisito.
<code>description</code>	document	Testo descrittivi del contesto normativo (<code>norm_description</code> , <code>target_description</code>).
<code>root_node</code>	document	Punto d'ingresso dell'albero decisionale per questo requisito.
<code>dependencies</code>	array	Elenco di codici di requisiti propedeutici da soddisfare prima della valutazione.

Tabella 8: Schema dati: Sub-documento — requirements[N]

Sub-documento --- DecisionNode

Campo	Tipo BSON	Descrizione
<code>id</code>	string	Codice identificativo del nodo logico.
<code>description</code>	string	Testo della domanda posta all'utente.
<code>child_yes</code>	document	Riferimento al nodo successivo sul ramo associato alla risposta affermativa.
<code>child_no</code>	document	Riferimento al nodo successivo sul ramo associato alla risposta negativa.

Tabella 9: Schema dati: Sub-documento — DecisionNode

Sub-documento --- LeafNode

Campo	Tipo BSON	Descrizione
<code>result</code>	enum	Stato terminale del percorso logico. Valori ammessi: <code>PASS</code> , <code>FAIL</code> , <code>NA</code> .
<code>not_applicable</code>	string	Testo esplicativo in caso di non applicabilità.
<code>pass</code>	string	Messaggio di conferma del soddisfacimento.
<code>fail</code>	string	Messaggio di errore o indicazioni di mitigazione.

Tabella 10: Schema dati: Sub-documento — LeafNode

3.6.4) Gestione degli Esiti e Storicità

La separazione netta tra Modello e Dispositivo ottimizza la memorizzazione e la gestione dei dati portando i seguenti vantaggi:

- **Zero Ridondanza:** Il documento Dispositivo salva unicamente il dizionario delle risposte fornite, mentre il documento Modello conserva esclusivamente la struttura statica dell'albero. Questo evita di duplicare l'intera gerarchia normativa per ogni dispositivo.
- **Disaccoppiamento:** Viene lasciato alla logica applicativa il compito di «fondere» i dati, compilando a runtime il template del Modello con le risposte storicizzate nel Dispositivo.
- **Versionamento:** La collection Modelli conserva tutte le versioni rilasciate di uno standard. Poiché il Dispositivo punta esplicitamente a una singola e specifica versione, lo storico delle certificazioni passate rimane inalterato e coerente anche a fronte di futuri aggiornamenti normativi.

4) Design Patterns

4.1) Principi di Design

4.1.1) Dependency Injection

Descrizione del pattern La dependency injection prevede che le dipendenze necessarie a una classe o a un modulo non vengano create internamente, ma fornite dall'esterno.

Generalmente questo avviene alla costruzione, tramite un setter o come parametro di un metodo.

Motivazioni dell'utilizzo del pattern La dependency injection permette di realizzare la Dependency Inversion e garantisce che le classi di dominio non dipendano mai da servizi secondari, framework o tecnologie esterne.

Utilizzo del pattern nel progetto All'interno del progetto viene usata la Constructor Injection. Il factory di Flask (`create_app()`) funge da **Composition Root**: è l'unico punto centralizzato del sistema a conoscere le implementazioni concrete delle porte e ad occuparsi di iniettarle nei service applicativi.

4.2) Pattern Architetture

4.2.1) Repository

Descrizione del pattern Il Repository è un pattern architetturale che astrae il sistema di persistenza dietro un'interfaccia che simula una collezione in memoria. Chi usa il Repository non conosce i dettagli del database sottostante.

Interagisce solo con metodi semantici come `findById()`, `save()`, `findAll()`.

Motivazioni dell'utilizzo del pattern Senza Repository la logica di business sarebbe mescolata con le query.

Inoltre i test unitari richiederebbero un database.

Con Repository il dominio e i service applicativi dipendono solo dall'interfaccia nei test si usa un mock.

Utilizzo del pattern nel progetto Nell'ambito del progetto viene utilizzata per gestire le comunicazioni con sistema di persistenza.

La lettura delle collection presenti su mongo DB avviene tramite porte dedicate.

DeviceRepositoryPort implementata da MongoDeviceRepository
e ComplianceStandardRepositoryPort implementata da
MongoComplianceStandardRepository

4.2.2) Unit of Work

Descrizione del pattern La Unit of Work è un pattern architetturale che traccia tutte le modifiche effettuate durante un'operazione e le applica al sistema di persistenza in modo atomico.

Se qualcosa va storto durante la scrittura, nessuna modifica parziale viene persistita.

Motivazioni dell'utilizzo del pattern Una sessione di valutazione può coinvolgere decine di modifiche e risposte ai nodi dei decision tree, aggiornamenti di stato, giustificazioni.

Scrivere ogni modifica immediatamente su MongoDB esporrebbe il sistema a stati inconsistenti se l'utente abbandona la sessione a metà.

La Unit of Work garantisce che il documento persistito sia sempre in uno stato coerente.

Utilizzo del pattern nel progetto Nel progetto, questo pattern regola il ciclo di sessione. Il ValutazioneService accumula e gestisce le modifiche di stato direttamente in memoria durante la navigazione.

Il documento su MongoDB non viene mai toccato o aggiornato durante i passaggi intermedi: la scrittura sul database avviene una sola volta, in modo atomico tramite il Repository, solo quando si raggiunge un nodo foglia o quando il progresso intermedio viene esplicitamente confermato.

4.3) Design Patterns Creazionali

4.3.1) Factory

Descrizione del pattern Il pattern Factory è un pattern creazionale che astrae il processo di creazione degli oggetti.

Delega a un componente specifico la responsabilità di creare oggetti complessi, restituendoli attraverso un'interfaccia comune.

Motivazioni dell'utilizzo del pattern In un'architettura esagonale, i layer di dominio devono dipendere da interfacce (Porte) e mai da implementazioni concrete.

Se un service dovesse istanziare direttamente le proprie dipendenze, finirebbe per accoppiarsi a librerie o tecnologie esterne.

Il Factory risolve questo problema incapsulando la conoscenza di «come» e «quale» oggetto concreto creare.

Utilizzo del pattern nel progetto Nel progetto il pattern è applicato a due livelli distinti:

1. **Application Factory:** A livello di framework, la funzione `create_app()` di Flask agisce come macro-factory, assemblando l'applicazione e le sue dipendenze principali all'avvio.

2. **Domain/Service Factory:** A livello applicativo, vengono utilizzate factory specifiche per costruire a runtime le implementazioni corrette di alcune interfacce che richiedono una selezione basata sulle azioni dell'utente (creare il FileExporter corrispondente al formato richiesto dall'utente), restituendo ai service oggetti pronti all'uso che rispettano l'interfaccia attesa.

4.4) Design Patterns strutturali

4.4.1) Adapter

Descrizione del pattern

L'Adapter è un pattern strutturale che traduce l'interfaccia di un componente esterno nell'interfaccia che il sistema si aspetta.

Permette a due componenti incompatibili di collaborare senza che nessuno dei due debba cambiare la propria interfaccia.

Motivazioni dell'utilizzo del pattern Il dominio definisce le proprie interfacce in termini di concetti di business.

Tramite l'uso sistematico di porte e adapter è possibile ridurre l'accoppiamento permettendo l'aggiornamento parallelo dei sistemi e lasciando all'adapter il compito di tradurre le nuove interfacce

Utilizzo del pattern nel progetto Al fine di implementare correttamente l'architettura esagonale nel sistema backend, il pattern adapter è stato usato in modo sistematico per gestire le comunicazioni verso servizi esterni.

Gli utilizzi principali sono nella comunicazione col database e con i servizi di interpretazione e generazione di file

4.4.2) Facade

Descrizione del pattern Il Facade è un pattern strutturale che fornisce un'interfaccia di alto livello e semplificata a un insieme di componenti complessi o a un intero sottosistema. L'obiettivo è nascondere la complessità interna esponendo al chiamante unicamente i metodi strettamente necessari all'azione richiesta.

Motivazioni dell'utilizzo del pattern In un'architettura a livelli, i Facade evitano che i layer esterni (come i controller o le interfacce utente) debbano orchestrare manualmente le dipendenze e la logica applicativa. Questo permette di mantenere i moduli di ingresso «sottili» e disaccoppiati dalle dinamiche interne del dominio.

Utilizzo del pattern nel progetto Nel progetto, questo pattern si riflette nei Service, come ad esempio il FileImporterService. Anche se espone un'interfaccia minimale, agisce da Facade orchestrando diverse operazioni sottostanti: riceve la richiesta, coordina i servizi di estrazione e parsing del file, e istruisce il sistema di persistenza per il salvataggio, nascondendo l'intera complessità della sequenza al chiamante.

4.5) Design Patterns Comportamentali

4.5.1) Strategy

Descrizione del pattern Lo Strategy è un pattern comportamentale che definisce una famiglia di algoritmi intercambiabili, incapsulandoli in classi separate che condividono la stessa interfaccia. Il chiamante seleziona quale algoritmo usare a runtime senza modificare il proprio codice.

Motivazioni dell'utilizzo del pattern Il sistema deve supportare più formati di import ed export — JSON, CSV, XML — e potenzialmente nuovi formati in futuro. Con Strategy supportare un nuovo formato consiste nel creare una nuova classe che implementa lo strategy pattern senza modificare il codice già esistente.

Utilizzo del pattern nel progetto Viene utilizzato nel processo di importazione del file

4.5.2) Template Method

Descrizione del pattern Il Template Method è un pattern comportamentale che definisce lo scheletro di un algoritmo in una classe astratta, delegando alle sottoclassi l'implementazione dei passi specifici.

La sequenza complessiva è fissa; ciò che cambia è il comportamento dei singoli passi.

Motivazioni dell'utilizzo del pattern Tutti i formati di export condividono la stessa sequenza operativa: preparare la struttura, scrivere i dati, finalizzare l'output.

Senza Template Method questa sequenza dovrebbe essere replicata in ogni exporter concreto, introducendo duplicazione e rischio di inconsistenze.

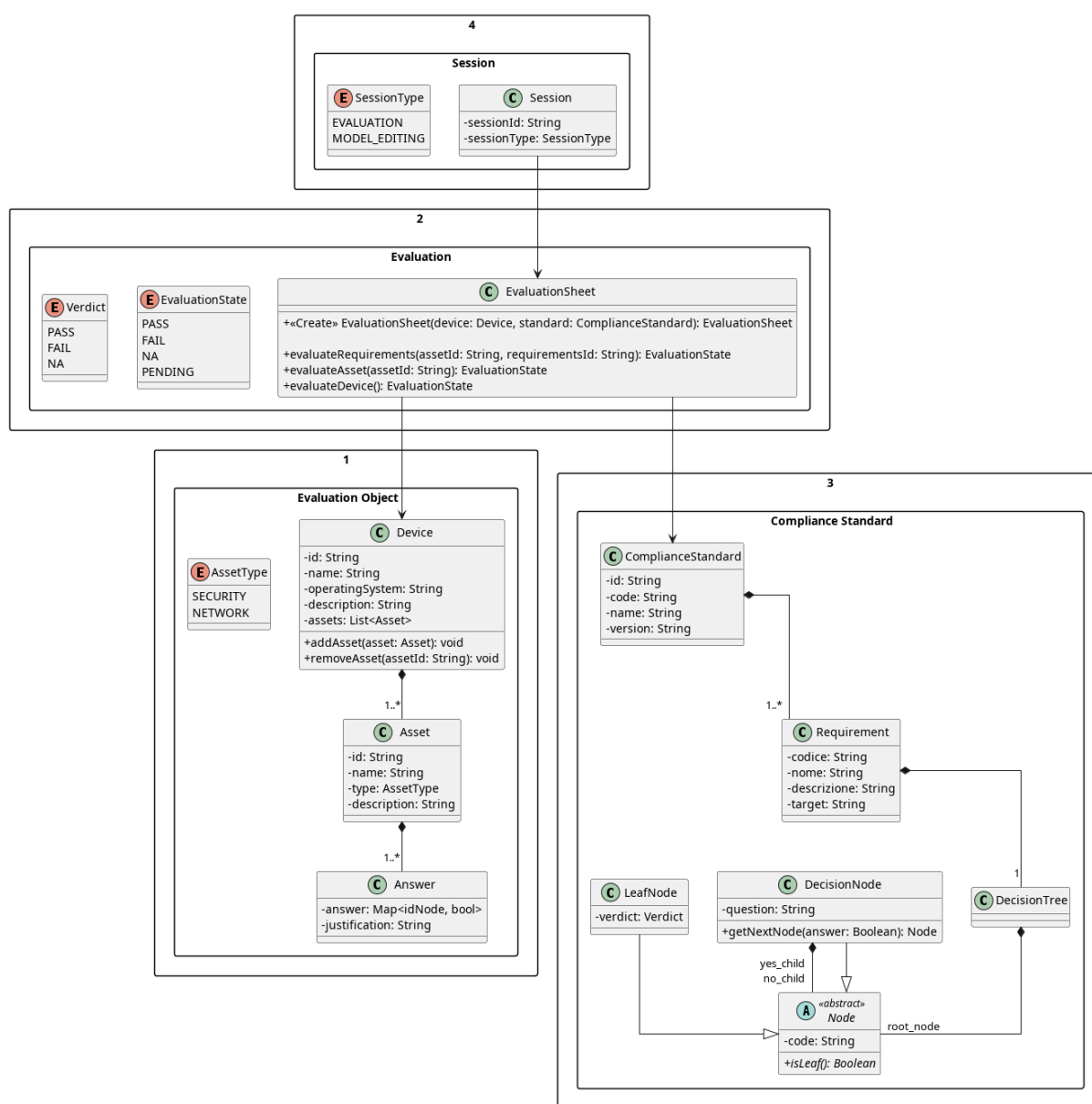
Il Template Method consente di scrivere la sequenza una sola volta nella classe astratta, lasciando alle sottoclassi solo i dettagli specifici del formato.

Utilizzo del pattern nel progetto Viene utilizzato durante l'esportazione di informazioni sotto formato di file in diversi formati.

5) Diagrammi delle classi

5.1) Diagramma di dominio

Il diagramma delle classi illustra il Modello di Dominio del nucleo applicativo. Progettato rispettando i principi del Domain-Driven Design e dell'Architettura Esagonale, il modello incapsula esclusivamente la logica di business pura, risultando del tutto agnostico rispetto ai dettagli infrastrutturali.



Per garantire coerenza logica e separazione delle responsabilità, le classi sono state logicamente organizzate in tre macro-package:

1. Oggetti di Valutazione

Questo package modella le entità concrete sottoposte alla valutazione:

- **Dispositivo**: mantiene lo stato globale dell'oggetto in esame e gestisce il ciclo di vita delle sue componenti.
- **Asset**: rappresenta i singoli componenti fisici o logici (classificati tramite l'enumerazione `TipoAsset` in `Security` o `Network`) che costituiscono il dispositivo.

La relazione tra **Dispositivo** e **Asset** è una composizione. La distruzione logica di un dispositivo all'interno del sistema comporta la distruzione dei relativi asset associati.

2. Modello Normativo

Questo package incapsula la struttura formale della norma di riferimento.

- **ModelloStandard** e **Requisito**: Il **ModelloStandard** definisce l'anagrafica della norma e si compone di molteplici **Requisiti**. La relazione riflessiva su **Requisito** («dipende da») permette di modellare i vincoli di dipendenza tra requisiti.
- **DecisionTree**: la struttura dell'albero di valutazione è stata modellata sfruttando il Design Pattern Composite. La classe astratta **NodoNormativo** definisce il contratto base, che viene implementato polimorficamente da:
 - **NodoDecisionale**: nodi intermedi che incapsulano una domanda e definiscono una biforcazione logica (YES/NO) verso i nodi successivi.
 - **NodoFoglia**: nodi terminali del ramo decisionale che emettono un **Verdetto** di conformità (PASS, FAIL, NA) per l'albero a cui appartengono

3. Navigazione e Valutazione

Questo package funge da area di interazione tra l'entità fisica e la regola normativa.

- **ValutazioneRequisito**: Agisce come Classe di Associazione tra uno specifico **Asset** e uno specifico **Requisito**. Questa scelta progettuale separa nettamente la definizione della norma dallo stato della valutazione corrente. Contiene una struttura dati (`mapRisposte`) fondamentale per memorizzare le risposte fornite dall'utente durante la navigazione del **DecisionTree**, garantendo la possibilità di sospendere e riprendere la compilazione.
- **ReportConformita**: È legato a **ValutazioneRequisito** tramite una relazione di aggregazione: il report colleziona le singole valutazioni per generare l'esito finale, ma una sua eventuale eliminazione o rigenerazione non invalida i dati delle valutazioni persistite nel sistema.

Viene fatto utilizzo di enumerazioni (`TipoAsset`, `StatoValutazione`, `Verdetto`). Questa scelta impone vincoli di dominio stringenti, garantendo la type-safety ed evitando stati di valutazione non previsti dal capitolato.

5.2) Device

5.2.1) WriteDeviceModule

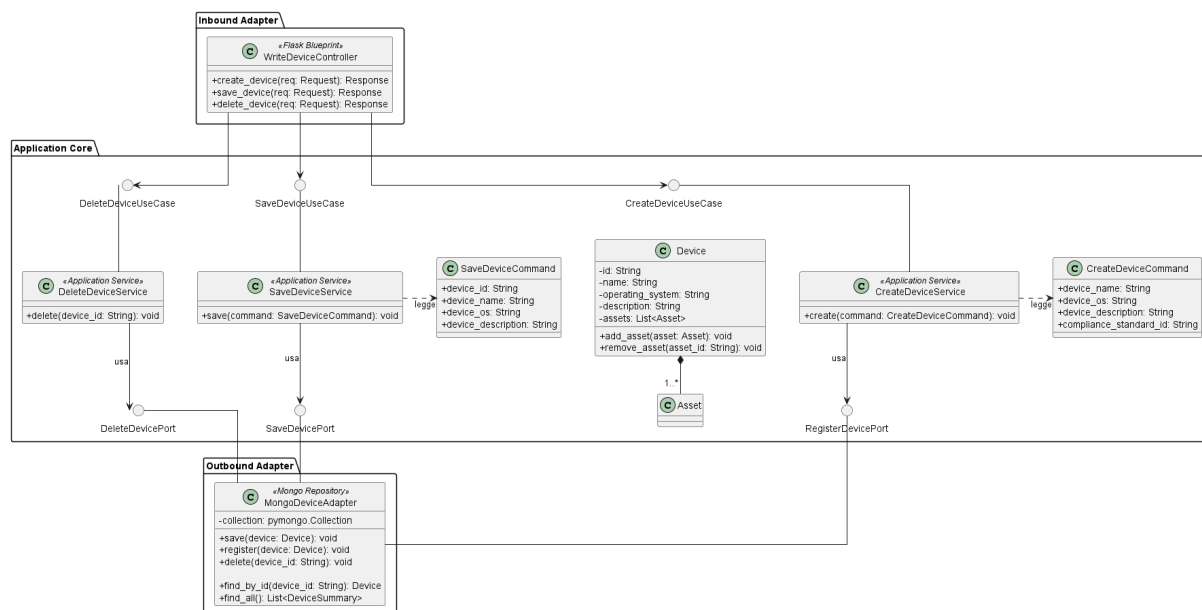


Figura 3: Modulo di scrittura Dispositivi

Il diagramma offre una visione d'insieme del modulo di scrittura per la gestione dei Dispositivi, mostrando come i tre casi d'uso — *CreateDevice*, *SaveDevice* e *DeleteDevice* — condividano gli stessi componenti infrastrutturali (*WriteDeviceController* e *MongoDeviceAdapter*) pur introducendo ciascuno le proprie interfacce e service dedicati. I componenti sono descritti in dettaglio nelle sezioni seguenti.

5.2.2) CreateDevice

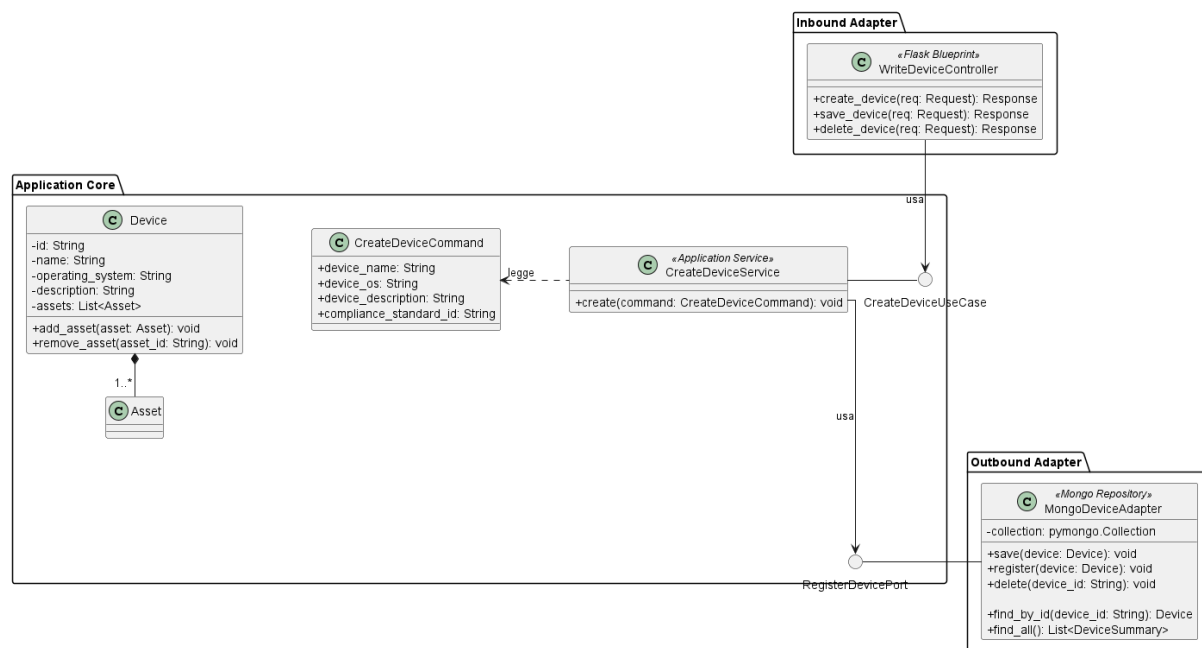


Figura 4: Caso d'uso CreateDevice

Il diagramma illustra l'architettura del modulo di scrittura per la gestione dei Dispositivi, coprendo le operazioni di creazione, modifica ed eliminazione secondo i principi dell'architettura esagonale.

5.2.2.1) WriteDeviceController



Figura 5: WriteDeviceController

Descrizione

WriteDeviceController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP relative alla gestione dei Dispositivi e le inoltra al livello applicativo.

Attributi

WriteDeviceController non definisce attributi propri.

Metodi e funzioni

- + `create_device(req: Request): Response` — riceve la richiesta HTTP di creazione di un nuovo Dispositivo, estrae i dati dal corpo della richiesta e li inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.
- + `save_device(req: Request): Response` — riceve la richiesta HTTP di modifica di un Dispositivo esistente, estrae i dati aggiornati e li inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.
- + `delete_device(req: Request): Response` — riceve la richiesta HTTP di eliminazione di un Dispositivo, estrae l'identificativo dalla richiesta e lo inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.

5.2.2.2) CreateDeviceUseCase

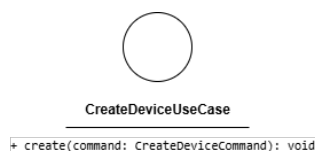


Figura 6: CreateDeviceUseCase

Descrizione

CreateDeviceUseCase è l'interfaccia (Inbound Port) che definisce il contratto per la creazione di un nuovo Dispositivo. Viene implementata da *CreateDeviceService* e utilizzata da *WriteDeviceController*.

Attributi

CreateDeviceUseCase non definisce attributi.

Metodi e funzioni

- + `create(command: CreateDeviceCommand): void` — firma del metodo delegato all'esecuzione della logica di creazione a partire dai dati contenuti nel comando.

5.2.2.3) CreateDeviceCommand

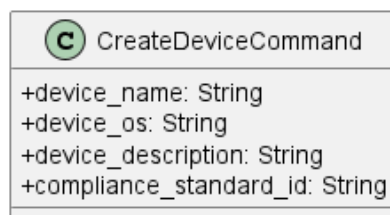


Figura 7: CreateDeviceCommand

Descrizione

CreateDeviceCommand è il Command Object che veicola i dati necessari alla creazione di un Dispositivo dal controller al service. L'uso di questo pattern separa la struttura dei dati in ingresso dall'entità di dominio, rendendo esplicita l'intenzione dell'operazione.

Attributi

- + `device_name: String` — nome del nuovo Dispositivo.
- + `device_os: String` — sistema operativo del Dispositivo.
- + `device_description: String` — descrizione testuale del Dispositivo.
- + `compliance_standard_id: String` — identificativo dello standard di conformità associato.

Metodi e funzioni

CreateDeviceCommand non definisce metodi.

5.2.2.4) CreateDeviceService



Figura 8: CreateDeviceService

Descrizione

CreateDeviceService è il service applicativo appartenente all'Application Core responsabile della logica di creazione di un Dispositivo. Implementa l'interfaccia *CreateDeviceUseCase*, riceve il comando in ingresso, costruisce l'entità di dominio e ne coordina la persistenza tramite *RegisterDevicePort*.

Attributi

- - register_device_port: RegisterDevicePort — porta outbound utilizzata per la persistenza del nuovo Dispositivo.

Metodi e funzioni

- + create(command: CreateDeviceCommand): void — concretizza il contratto definito da *CreateDeviceUseCase*. Mappa i dati del comando nell'entità *Device* e ne richiede la registrazione tramite *RegisterDevicePort*.

5.2.2.5) Device

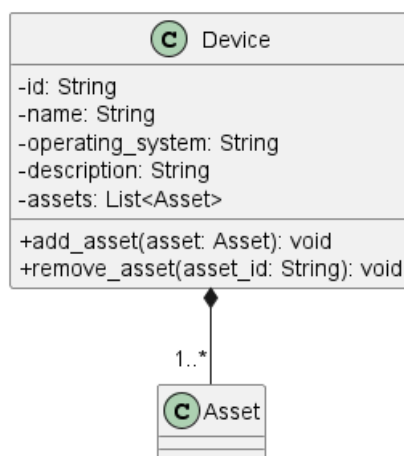


Figura 9: Device

Descrizione

Device è l'entità centrale del dominio che rappresenta il Dispositivo oggetto della valutazione di conformità. È associata alla classe *Asset* con una relazione di composizione avente cardinalità 1..*.

Attributi

- - id: String — identificativo univoco del Dispositivo.
- - name: String — nome del Dispositivo.
- - operating_system: String — sistema operativo.
- - description: String — descrizione testuale.
- - assets: List<Asset> — lista degli asset associati al Dispositivo.

Metodi e funzioni

- + add_asset(asset: Asset): void — aggiunge un *Asset* alla lista del Dispositivo.
- + remove_asset(asset_id: String): void — rimuove l'*Asset* identificato da `asset_id` dalla lista del Dispositivo.

5.2.2.6) RegisterDevicePort

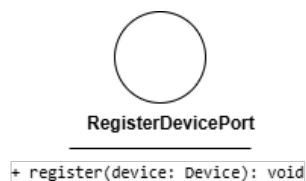


Figura 10: RegisterDevicePort

Descrizione

RegisterDevicePort è l'interfaccia (Outbound Port) che definisce il contratto per la registrazione di un nuovo Dispositivo nel sistema di persistenza. Viene implementata da *MongoDeviceAdapter* e utilizzata da *CreateDeviceService*.

Attributi

RegisterDevicePort non definisce attributi.

Metodi e funzioni

- + register(device: Device): void — firma del metodo che esegue l'inserimento fisico del Dispositivo nel sistema di persistenza.

5.2.2.7) MongoDeviceAdapter



Figura 11: MongoDeviceAdapter

Descrizione

MongoDeviceAdapter è la classe dell'Outbound Adapter che implementa le porte outbound del sistema, traducendo le operazioni del dominio in interazioni concrete con MongoDB tramite `pymongo.Collection`.

Attributi

- - collection: pymongo.Collection — riferimento alla collezione MongoDB su cui vengono eseguite le operazioni di persistenza.

Metodi e funzioni

- + `save(device: Device): void` — salva le modifiche a un Dispositivo esistente nella collezione.
- + `register(device: Device): void` — inserisce un nuovo Dispositivo nella collezione.
- + `delete(device_id: String): void` — rimuove il Dispositivo identificato da `device_id` dalla collezione.
- + `find_by_id(device_id: String): Device` — recupera il Dispositivo corrispondente all'identificativo fornito.
- + `find_all(): List<DeviceSummary>` — recupera la lista sintetica di tutti i Dispositivi presenti nella collezione.

5.2.3) DeleteDevice

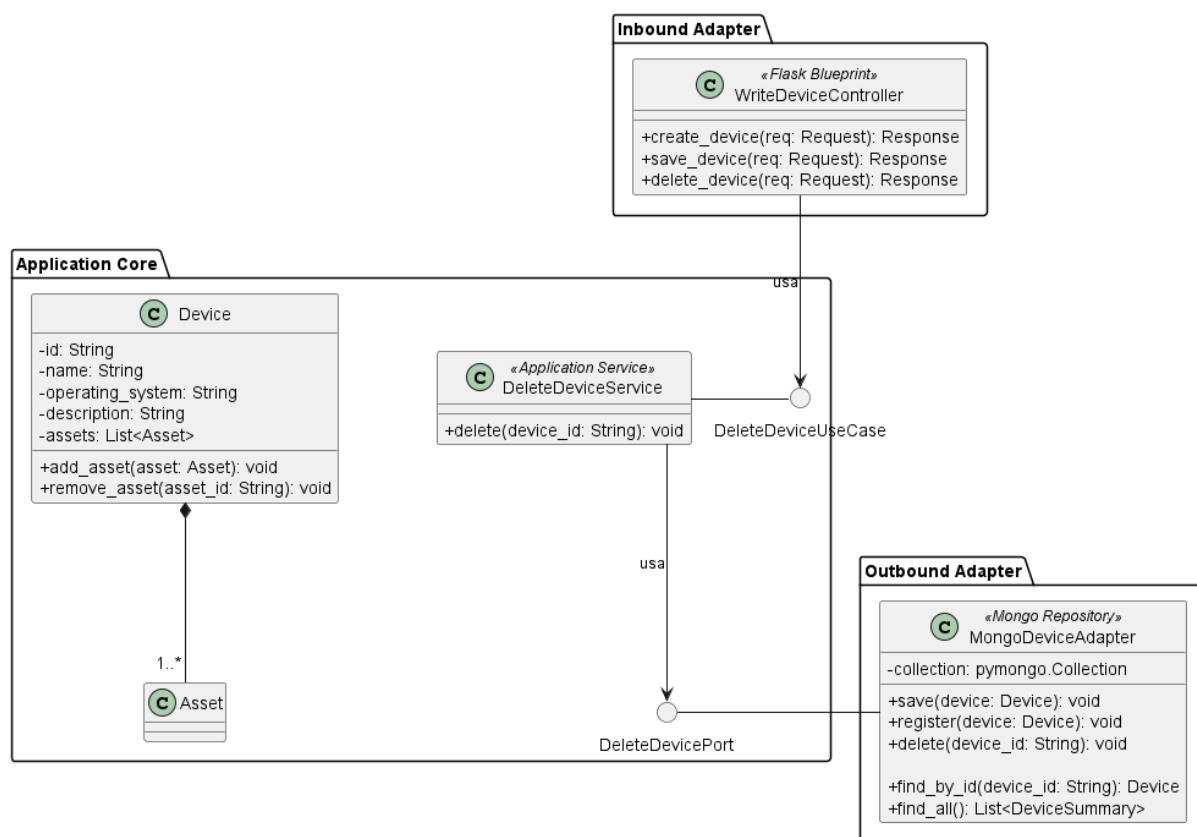


Figura 12: Caso d'uso DeleteDevice

Il diagramma illustra l'architettura del modulo dedicato all'eliminazione di un Dispositivo.

- Per la definizione di `WriteDeviceController`, vedere la sezione **Sezione 5.2.2.1**.
- Per la definizione di `Device`, vedere la sezione **Sezione 5.2.2.5**.
- Per la definizione di `MongoDeviceAdapter`, vedere la sezione **Sezione 5.2.2.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.2.3.1) DeleteDeviceUseCase

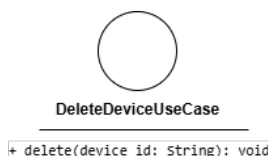


Figura 13: DeleteDeviceUseCase

Descrizione

DeleteDeviceUseCase è l'interfaccia (Inbound Port) che definisce il contratto per l'eliminazione di un Dispositivo. Viene implementata da *DeleteDeviceService* e utilizzata da *WriteDeviceController*.

Attributi

DeleteDeviceUseCase non definisce attributi.

Metodi e funzioni

- + delete(device_id: String): void — firma del metodo delegato all'esecuzione della logica di eliminazione a partire dall'identificativo del Dispositivo.

5.2.3.2) DeleteDeviceService



Figura 14: DeleteDeviceService

Descrizione

DeleteDeviceService è il service applicativo appartenente all'Application Core responsabile della logica di eliminazione di un Dispositivo. Implementa l'interfaccia *DeleteDeviceUseCase* e, a differenza del servizio di creazione, non richiede un Command Object: riceve direttamente l'identificativo del Dispositivo e ne coordina la rimozione dal sistema.

Attributi

DeleteDeviceService non definisce attributi propri.

Metodi e funzioni

- + delete(device_id: String): void — riceve l'identificativo univoco del Dispositivo e ne coordina la rimozione tramite *DeleteDevicePort*.

5.2.3.3) DeleteDevicePort

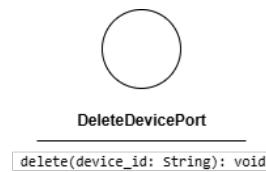


Figura 15: DeleteDevicePort

Descrizione

DeleteDevicePort è l'interfaccia (Outbound Port) che definisce il contratto per la rimozione fisica di un Dispositivo dal sistema di persistenza. Viene implementata da *MongoDeviceAdapter* e utilizzata da *DeleteDeviceService*.

Attributi

DeleteDevicePort non definisce attributi.

Metodi e funzioni

- + `delete(device_id: String): void` — firma del metodo che esegue la rimozione fisica del Dispositivo identificato da `device_id` dal sistema di persistenza.

5.2.4) SaveDevice

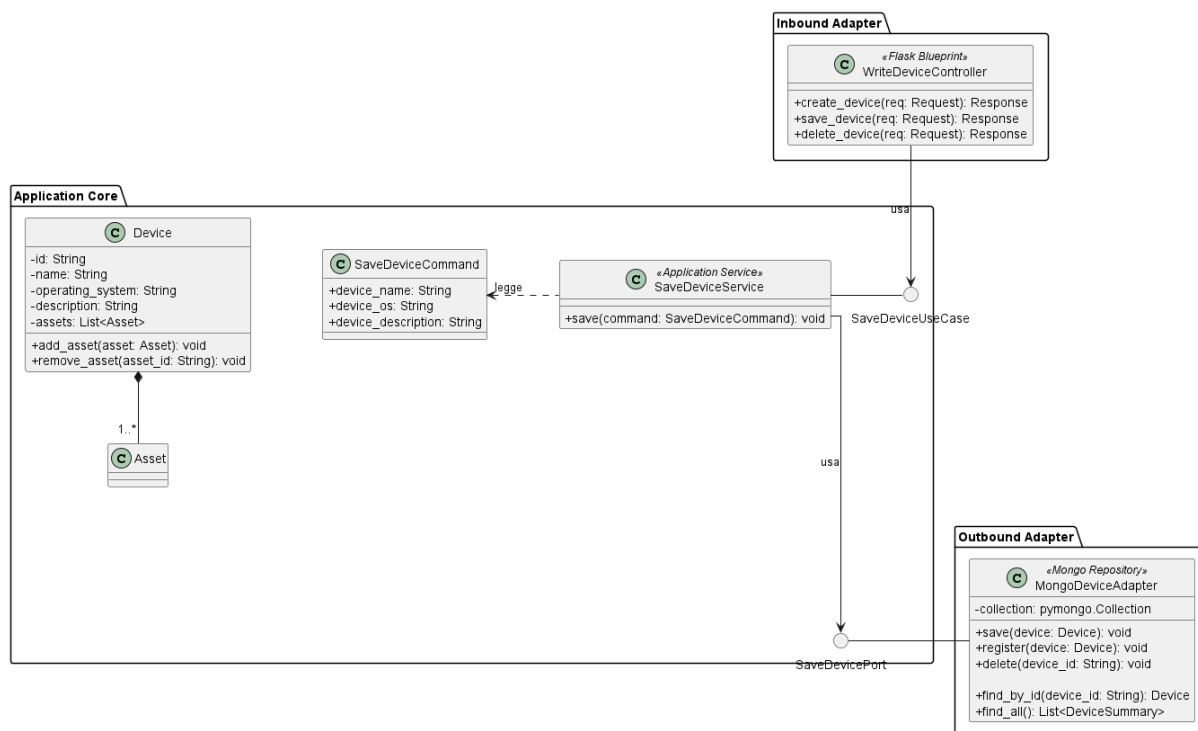


Figura 16: Caso d'uso SaveDevice

Il diagramma illustra l'architettura del modulo dedicato alla modifica e al salvataggio dello stato di un Dispositivo esistente.

- Per la definizione di *WriteDeviceController*, vedere la sezione **Sezione 5.2.2.1**.
- Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5**.
- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.2.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.2.4.1) SaveDeviceUseCase

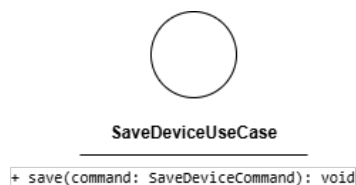


Figura 17: SaveDeviceUseCase

Descrizione

SaveDeviceUseCase è l'interfaccia (Inbound Port) che definisce il contratto per la modifica di un Dispositivo esistente. Viene implementata da *SaveDeviceService* e utilizzata da *WriteDeviceController*.

Attributi

SaveDeviceUseCase non definisce attributi.

Metodi e funzioni

- + save(command: SaveDeviceCommand): void — firma del metodo delegato all'esecuzione della logica di aggiornamento a partire dai dati contenuti nel comando.

5.2.4.2) SaveDeviceCommand

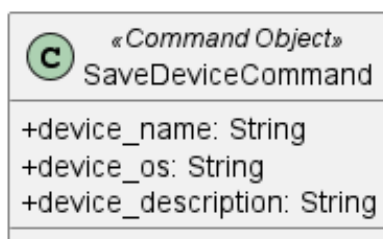


Figura 18: SaveDeviceCommand

Descrizione

SaveDeviceCommand è il Command Object che veicola i dati necessari alla modifica di un Dispositivo dal controller al service. Analogamente a *CreateDeviceCommand*, separa la struttura dei dati in ingresso dall'entità di dominio.

Attributi

- + device_name: String — nome aggiornato del Dispositivo.
- + device_os: String — sistema operativo aggiornato.
- + device_description: String — descrizione testuale aggiornata.

Metodi e funzioni

SaveDeviceCommand non definisce metodi.

5.2.4.3) SaveDeviceService

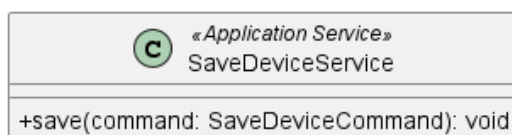


Figura 19: SaveDeviceService

Descrizione

SaveDeviceService è il service applicativo appartenente all'Application Core responsabile della logica di aggiornamento di un Dispositivo esistente. Implementa l'interfaccia *SaveDeviceUseCase*, riceve il comando in ingresso e ne coordina la persistenza tramite *SaveDevicePort*.

Attributi

SaveDeviceService non definisce attributi propri.

Metodi e funzioni

- `+ save(command: SaveDeviceCommand): void` — concretizza il contratto definito da *SaveDeviceUseCase*. Mappa i dati del comando nell'entità *Device* e ne richiede l'aggiornamento tramite *SaveDevicePort*.

5.2.4.4) SaveDevicePort

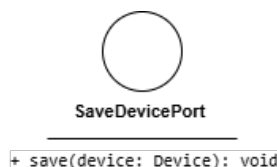


Figura 20: SaveDevicePort

Descrizione

SaveDevicePort è l'interfaccia (Outbound Port) che definisce il contratto per l'aggiornamento fisico di un Dispositivo nel sistema di persistenza. Viene implementata da *MongoDeviceAdapter* e utilizzata da *SaveDeviceService*.

Attributi

SaveDevicePort non definisce attributi.

Metodi e funzioni

- `+ save(device: Device): void` — firma del metodo che esegue l'aggiornamento fisico del Dispositivo nel sistema di persistenza.

5.2.5) ReadDeviceModule

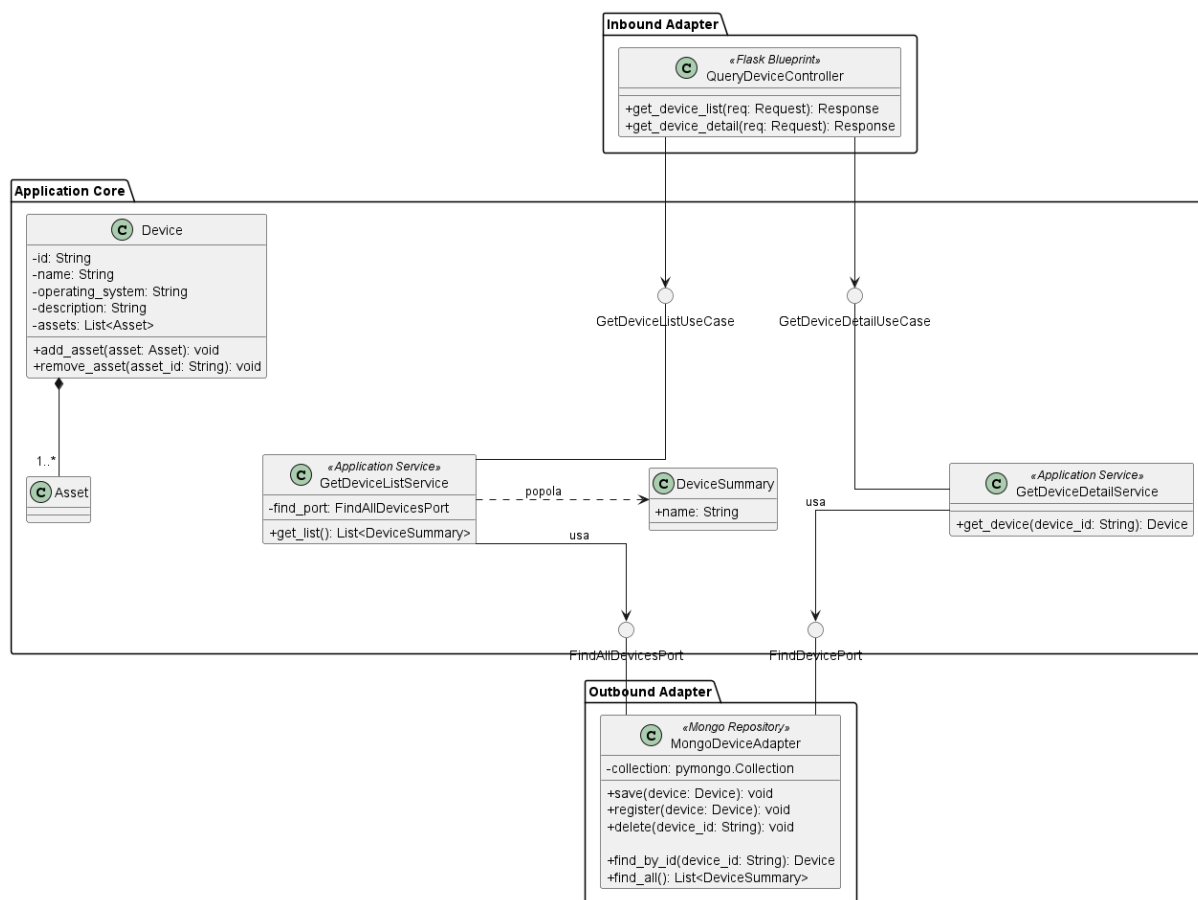


Figura 21: Modulo di lettura Dispositivi

Il diagramma illustra l'architettura del modulo di lettura per la gestione dei Dispositivi, coprendo le operazioni di recupero del dettaglio di un singolo Dispositivo e della lista sintetica di tutti i Dispositivi registrati.

- Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5**.
- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.2.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo modulo.

5.2.6) GetDeviceDetail

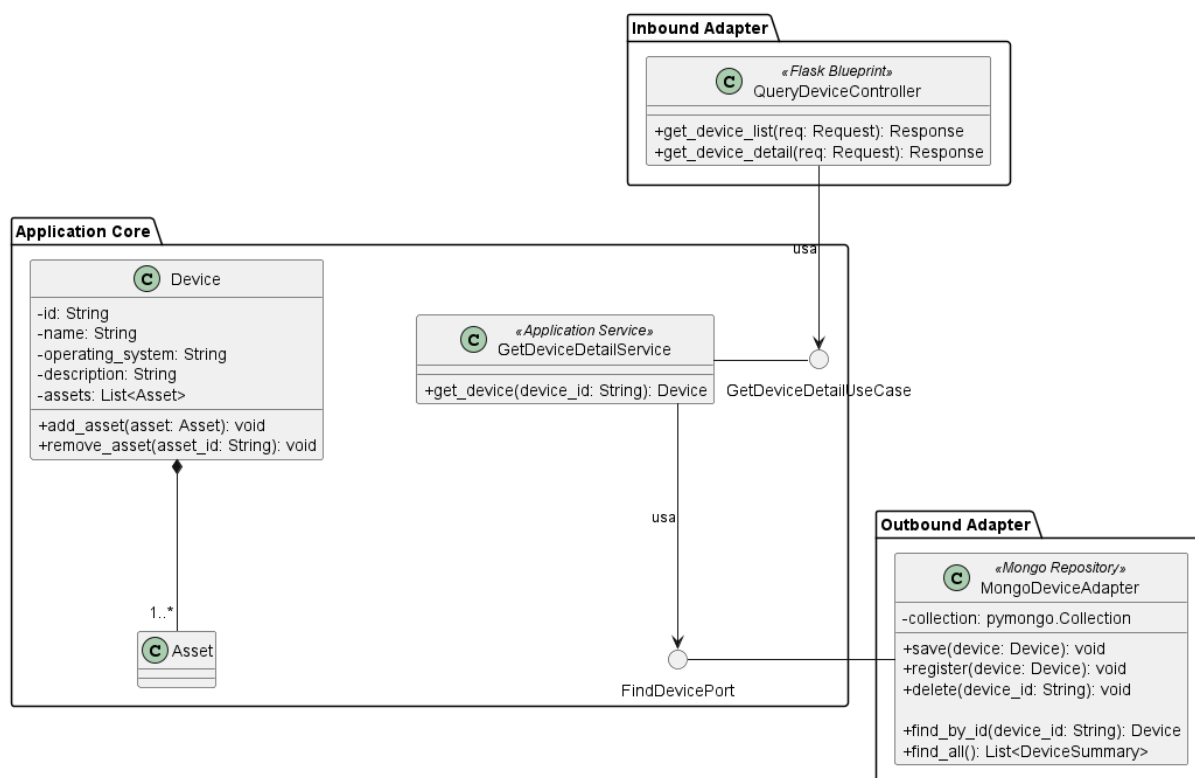


Figura 22: GetDeviceDetail

Il diagramma illustra l'architettura del modulo dedicato al recupero del dettaglio di un Dispositivo.

- Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5**.
- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.2.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.2.6.1) QueryDeviceController



Figura 23: QueryDeviceController

Descrizione

QueryDeviceController è il controller Flask appartenente all'*Inbound Adapter* che riceve le richieste HTTP di lettura relative ai Dispositivi e le inoltra al livello applicativo.

Attributi

QueryDeviceController non definisce attributi propri.

Metodi e funzioni

- + `get_device_list(req: Request): Response` — riceve la richiesta HTTP di recupero della lista dei Dispositivi e restituisce una risposta HTTP con l'elenco sintetico.
- + `get_device_detail(req: Request): Response` — riceve la richiesta HTTP di recupero del dettaglio di un Dispositivo specifico e restituisce una risposta HTTP con i dati completi.

5.2.6.2) GetDeviceDetailUseCase

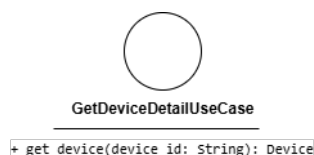


Figura 24: GetDeviceDetailUseCase

Descrizione

GetDeviceDetailUseCase è l'interfaccia (Inbound Port) che definisce il contratto per il recupero del dettaglio di un Dispositivo. Viene implementata da *GetDeviceDetailService* e utilizzata da *QueryDeviceController*.

Attributi

GetDeviceDetailUseCase non definisce attributi.

Metodi e funzioni

- + `get_device(device_id: String): Device` — firma del metodo delegato al recupero del Dispositivo corrispondente all'identificativo fornito.

5.2.6.3) GetDeviceDetailService



Figura 25: GetDeviceDetailService

Descrizione

GetDeviceDetailService è il service applicativo appartenente all'Application Core responsabile del recupero del dettaglio di un Dispositivo. Implementa l'interfaccia *GetDeviceDetailUseCase* e coordina il recupero dei dati tramite la porta outbound *FindDevicePort*.

Attributi

GetDeviceDetailService non definisce attributi propri.

Metodi e funzioni

- + `get_device(device_id: String): Device` — concretizza il contratto definito da *GetDeviceDetailUseCase*. Recupera il Dispositivo corrispondente all'identificativo fornito tramite *FindDevicePort*.

5.2.6.4) FindDevicePort

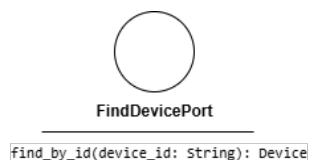


Figura 26: FindDevicePort

Descrizione

FindDevicePort è l'interfaccia (Outbound Port) che definisce il contratto per il recupero di un Dispositivo dal sistema di persistenza. Viene implementata da *MongoDeviceAdapter* e utilizzata da *GetDeviceDetailService*.

Attributi

FindDevicePort non definisce attributi.

Metodi e funzioni

- + `find_by_id(device_id: String): Device` — firma del metodo che recupera il Dispositivo corrispondente all'identificativo fornito dal sistema di persistenza.

5.2.7) GetDeviceList

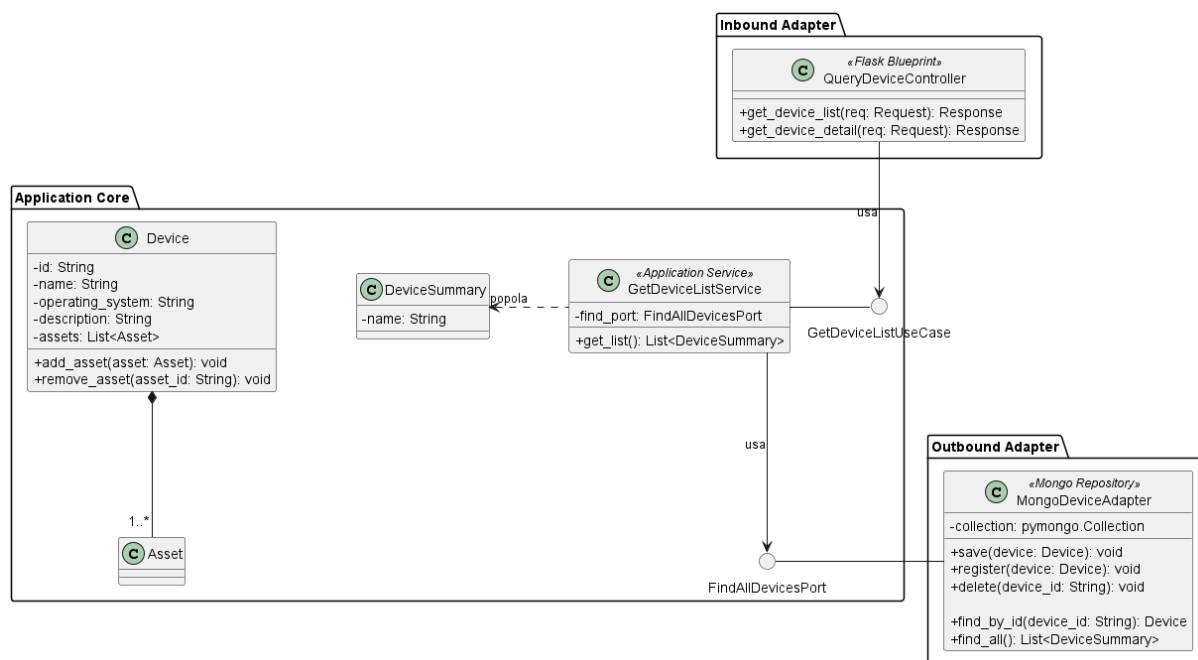


Figura 27: Caso d'uso GetDeviceList

Il diagramma illustra l'architettura del modulo dedicato al recupero della lista sintetica dei Dispositivi.

Per la definizione di *QueryDeviceController*, vedere la sezione **Sezione 5.2.6.1**.

Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5**.

Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.2.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.2.7.1) GetDeviceListUseCase

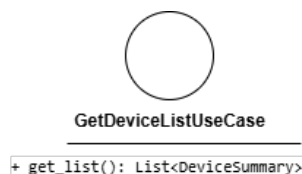


Figura 28: GetDeviceListUseCase

Descrizione

GetDeviceListUseCase è l'interfaccia (Inbound Port) che definisce il contratto per il recupero della lista sintetica dei Dispositivi. Viene implementata da *GetDeviceListService* e utilizzata da *QueryDeviceController*.

Attributi

GetDeviceListUseCase non definisce attributi.

Metodi e funzioni

- `+ get_list(): List<DeviceSummary>` — firma del metodo delegato al recupero della lista sintetica di tutti i Dispositivi presenti nel sistema.

5.2.7.2) GetDeviceListService

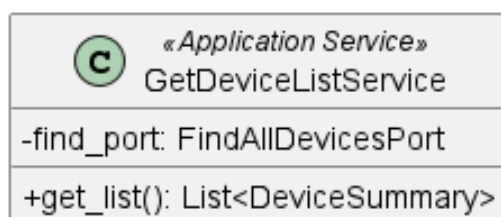


Figura 29: GetDeviceListService

Descrizione

GetDeviceListService è il service applicativo appartenente all'Application Core responsabile del recupero della lista sintetica dei Dispositivi. Implementa l'interfaccia *GetDeviceListUseCase* e coordina il recupero tramite la porta outbound *FindAllDevicesPort*.

Attributi

- `- find_port: FindAllDevicesPort` — porta outbound utilizzata per il recupero della lista dei Dispositivi dal sistema di persistenza.

Metodi e funzioni

- `+ get_list(): List<DeviceSummary>` — concretizza il contratto definito da *GetDeviceListUseCase*. Recupera la lista sintetica di tutti i Dispositivi tramite *FindAllDevicesPort*.

5.2.7.3) DeviceSummary

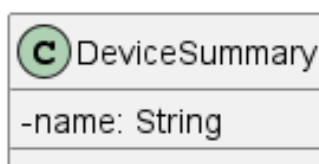


Figura 30: DeviceSummary

Descrizione

DeviceSummary è il Data Transfer Object che veicola la rappresentazione sintetica di un Dispositivo verso il livello di presentazione. Espone esclusivamente le informazioni necessarie alla visualizzazione in lista, evitando di esporre l'intera entità di dominio.

Attributi

- - name: String — nome del Dispositivo.

Metodi e funzioni

DeviceSummary non definisce metodi.

5.2.7.4) FindAllDevicesPort

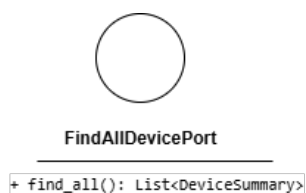


Figura 31: FindAllDevicesPort

Descrizione

FindAllDevicesPort è l'interfaccia (Outbound Port) che definisce il contratto per il recupero della lista sintetica di tutti i Dispositivi dal sistema di persistenza. Viene implementata da *MongoDeviceAdapter* e utilizzata da *GetDeviceListService*.

Attributi

FindAllDevicesPort non definisce attributi.

Metodi e funzioni

- + find_all(): List<DeviceSummary> — firma del metodo che recupera la lista sintetica di tutti i Dispositivi presenti nel sistema di persistenza.

5.3) Asset

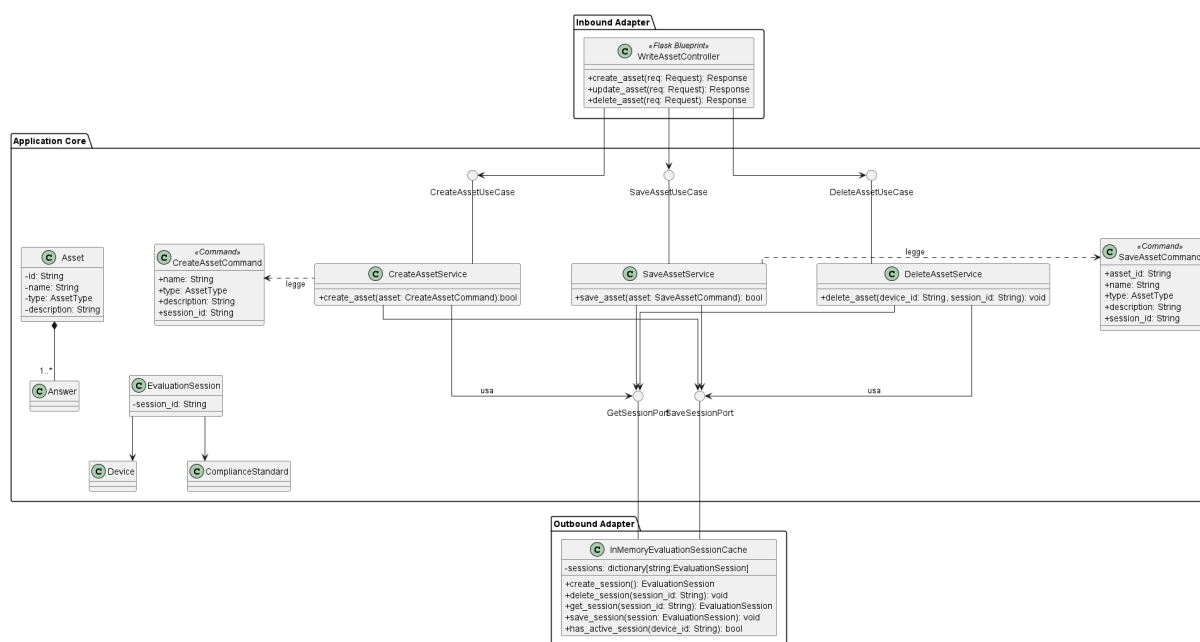


Figura 32: Modulo di scrittura Asset

Il diagramma illustra l'architettura del modulo di scrittura per la gestione degli Asset, coprendo le operazioni di creazione, modifica ed eliminazione secondo i principi dell'architettura esagonale.

Nei paragrafi seguenti vengono descritti in dettaglio i componenti di ciascun caso d'uso.

5.3.1) CreateAsset

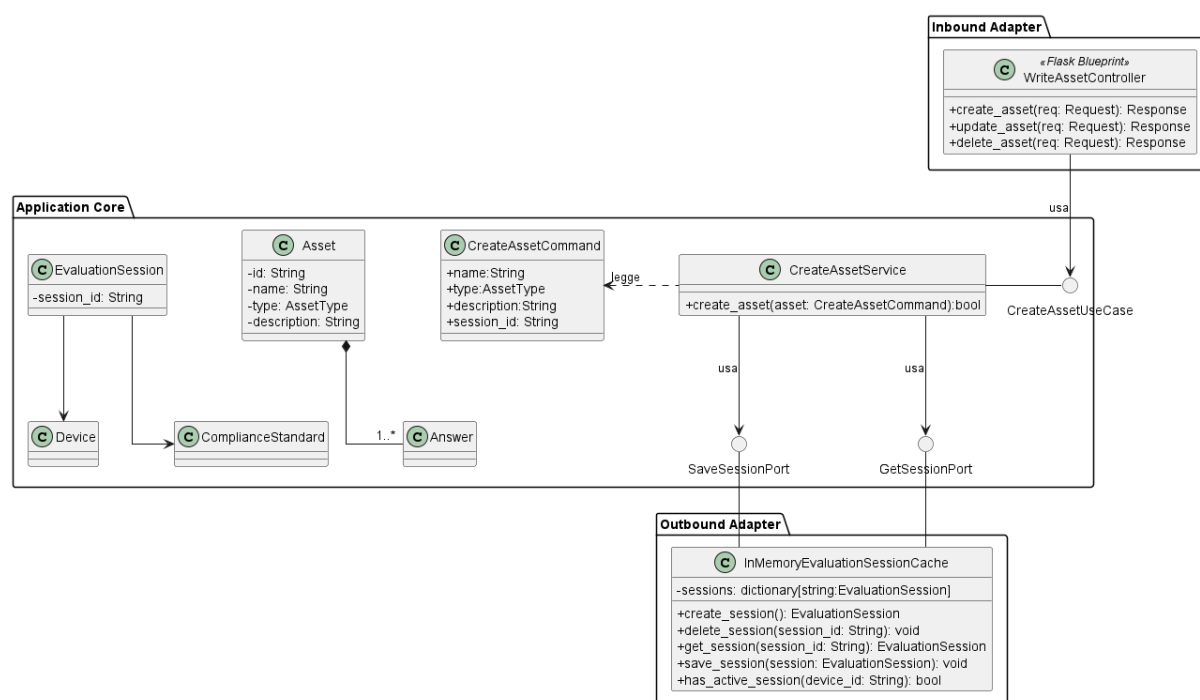


Figura 33: Caso d'uso CreateAsset

Il diagramma illustra l'architettura del modulo di creazione di un Asset all'interno di una sessione di valutazione attiva.

5.3.1.1) WriteAssetController



Figura 34: WriteAssetController

Descrizione

WriteAssetController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP relative alla gestione (scrittura) degli Asset e le inoltra al livello applicativo.

Attributi

WriteAssetController non definisce attributi propri.

Metodi e funzioni

- `+ create_asset(req: Request): Response` — riceve la richiesta HTTP di creazione di un nuovo Asset, estrae i dati dal corpo della richiesta e li inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.
- `+ update_asset(req: Request): Response` — riceve la richiesta HTTP di aggiornamento di un Asset esistente, estrae i dati modificati e li inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.
- `+ delete_asset(req: Request): Response` — riceve la richiesta HTTP di eliminazione di un Asset, estrae l'identificativo dalla richiesta e lo inoltra al livello applicativo; restituisce una risposta HTTP con l'esito dell'operazione.

5.3.1.2) CreateAssetUseCase

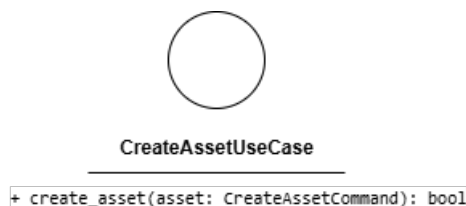


Figura 35: CreateAssetUseCase

Descrizione

CreateAssetUseCase è l'interfaccia (Inbound Port) che definisce il contratto per la creazione di un nuovo Asset all'interno della sessione di valutazione. Viene implementata da *CreateAssetService* e utilizzata da *WriteAssetController*.

Attributi

CreateAssetUseCase non definisce attributi.

Metodi e funzioni

- `+ create_asset(asset: CreateAssetCommand): bool` — firma del metodo delegato all'esecuzione della logica di creazione a partire dai dati contenuti nel comando.

5.3.1.3) CreateAssetCommand



Figura 36: CreateAssetCommand

Descrizione

CreateAssetCommand è il Command Object che veicola i dati necessari alla creazione di un Asset dal controller al service. Separa la struttura dei dati in ingresso dall'entità di dominio, rendendo esplicita l'intenzione dell'operazione.

Attributi

- `+ name: String` — nome del nuovo Asset.
- `+ type: AssetType` — tipo dell'Asset.
- `+ description: String` — descrizione testuale dell'Asset.
- `+ session_id: String` — identificativo della sessione di valutazione attiva a cui l'Asset viene associato.

Metodi e funzioni

CreateAssetCommand non definisce metodi.

5.3.1.4) CreateAssetService

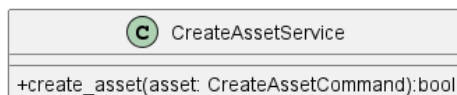


Figura 37: CreateAssetService

Descrizione

CreateAssetService è il service applicativo appartenente all'Application Core responsabile della logica di creazione di un Asset. Implementa l'interfaccia *CreateAssetUseCase*, recupera la sessione attiva tramite *GetSessionPort*, aggiunge il nuovo Asset e persiste la sessione aggiornata tramite *SaveSessionPort*.

Attributi

CreateAssetService non definisce attributi propri.

Metodi e funzioni

- `+ create_asset(asset: CreateAssetCommand): bool` — concretizza il contratto definito da *CreateAssetUseCase*. Recupera la sessione attiva, vi aggiunge il nuovo Asset e ne persiste lo stato aggiornato.

5.3.1.5) Asset

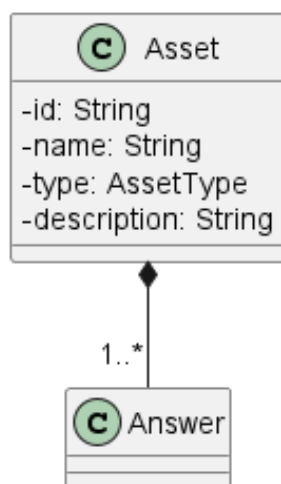


Figura 38: Asset

Descrizione

Asset è l'entità di dominio che rappresenta un asset oggetto di valutazione di conformità all'interno di una sessione. È associata a *ComplianceStandard* e ad *Answer* con cardinalità `1..*`.

Attributi

- - id: String — identificativo univoco dell'Asset.
- - name: String — nome dell'Asset.
- - type: AssetType — tipo dell'Asset.
- - description: String — descrizione testuale dell'Asset.

Metodi e funzioni

Asset non definisce metodi.

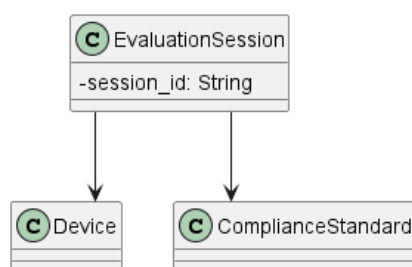
5.3.1.6) EvaluationSession

Figura 39: EvaluationSession

Descrizione

EvaluationSession è l'entità di dominio che aggrega gli Asset oggetto di valutazione e mantiene il riferimento alla sessione di valutazione corrente. È associata a *Device* e *ComplianceStandard*.

Attributi

- - session_id: String — identificativo univoco della sessione di valutazione.

Metodi e funzioni

EvaluationSession non espone metodi pubblici nel diagramma.

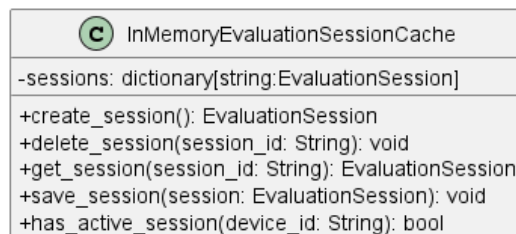
5.3.1.7) InMemoryEvaluationSessionCache

Figura 40: InMemoryEvaluationSessionCache

Descrizione

InMemoryEvaluationSessionCache è la classe dell'Outbound Adapter annotata come *Session Cache* che implementa entrambe le porte outbound *SaveSessionPort* e *GetSes-*

sessionPort. Gestisce la persistenza in memoria delle sessioni di valutazione tramite un dizionario indicizzato per *session_id*.

Attributi

- - *sessions*: `dictionary[string: EvaluationSession]` — struttura dati in memoria che mantiene le sessioni di valutazione attive, indicizzate per *session_id*.

Metodi e funzioni

- + *create_session()*: `EvaluationSession` — crea e registra una nuova sessione di valutazione.
- + *delete_session(session_id: String)*: `void` — rimuove la sessione identificata da *session_id*.
- + *get_session(session_id: String)*: `EvaluationSession` — recupera la sessione corrispondente all'identificativo fornito.
- + *save_session(session: EvaluationSession)*: `void` — aggiorna la sessione nel dizionario in memoria.
- + *has_active_session(device_id: String)*: `bool` — verifica se esiste una sessione attiva per il Dispositivo identificato da *device_id*.

5.3.1.8) SaveSessionPort



Figura 41: SaveSessionPort

Descrizione

SaveSessionPort è l'interfaccia (Outbound Port) che definisce il contratto per il salvataggio della sessione di valutazione nel sistema di persistenza in memoria. Viene implementata da *InMemoryEvaluationSessionCache* e utilizzata da tutti i service del modulo che modificano lo stato della sessione, tra cui *CreateAssetService*, *SaveAssetService* e *DeleteAssetService*.

Attributi

SaveSessionPort non definisce attributi.

Metodi e funzioni

- + *save_session(session: EvaluationSession)*: `void` — firma del metodo che persiste la sessione aggiornata nel sistema in memoria.

5.3.1.9) GetSessionPort



Figura 42: GetSessionPort

Descrizione

GetSessionPort è l'interfaccia (Outbound Port) che definisce il contratto per il recupero della sessione di valutazione dal sistema di persistenza in memoria. Viene implementata da *InMemoryEvaluationSessionCache* e utilizzata da tutti i service del modulo che necessitano di accedere alla sessione corrente, tra cui *CreateAssetService*, *SaveAssetService*, *DeleteAssetService* e *GetAssetDetailService*.

Attributi

GetSessionPort non definisce attributi.

Metodi e funzioni

- + `get_session(session_id: String): EvaluationSession` — firma del metodo che recupera la sessione di valutazione attiva corrispondente all'identificativo fornito.

5.3.2) SaveAsset

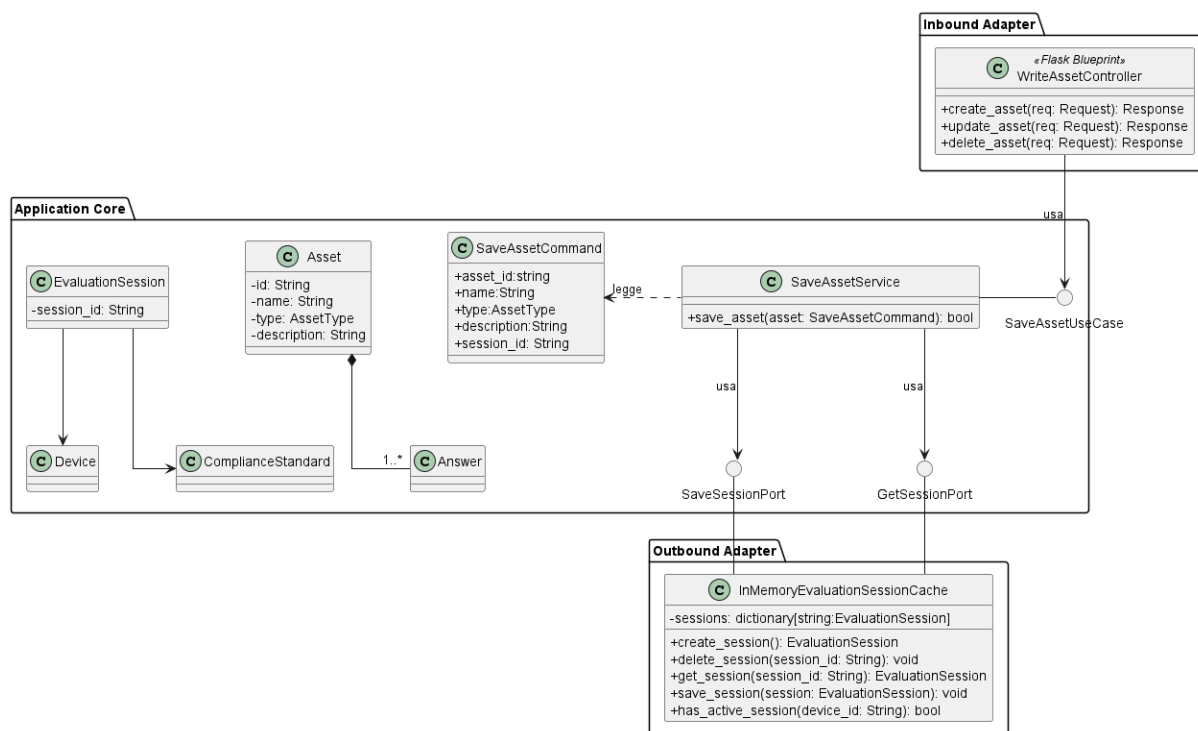


Figura 43: Caso d'uso SaveAsset

Il diagramma illustra l'architettura del modulo di modifica di un Asset esistente all'interno di una sessione di valutazione attiva.

- Per la definizione di *WriteAssetController*, vedere la sezione **Sezione 5.3.1.1.**
- Per la definizione di *Asset*, vedere la sezione **Sezione 5.3.1.5.**
- Per la definizione di *EvaluationSession*, vedere la sezione **Sezione 5.3.1.6.**
- Per la definizione di *SaveSessionPort*, vedere la sezione **Sezione 5.3.1.8.**
- Per la definizione di *GetSessionPort*, vedere la sezione **Sezione 5.3.1.9.**
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7.**

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.3.2.1) SaveAssetUseCase



Figura 44: SaveAssetUseCase

Descrizione

SaveAssetUseCase è l'interfaccia (Inbound Port) che definisce il contratto per la modifica di un Asset esistente all'interno della sessione di valutazione. Viene implementata da *SaveAssetService* e utilizzata da *WriteAssetController*.

Attributi

SaveAssetUseCase non definisce attributi.

Metodi e funzioni

- `+ save_asset(asset: SaveAssetCommand): bool` — firma del metodo delegato all'esecuzione della logica di aggiornamento a partire dai dati contenuti nel comando.

5.3.2.2) SaveAssetCommand

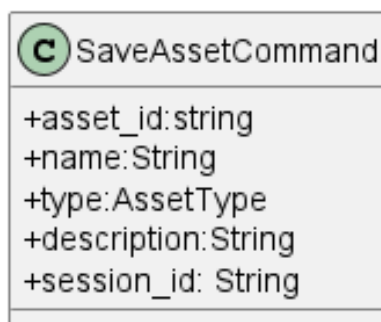


Figura 45: SaveAssetCommand

Descrizione

SaveAssetCommand è il Command Object che veicola i dati necessari alla modifica di un Asset dal controller al service. Separa la struttura dei dati in ingresso dall'entità di dominio.

Attributi

- `+ asset_id: String` — identificativo univoco dell'Asset da modificare.
- `+ name: String` — nome aggiornato dell'Asset.
- `+ type: AssetType` — tipo aggiornato dell'Asset.
- `+ description: String` — descrizione testuale aggiornata dell'Asset.
- `+ session_id: String` — identificativo della sessione di valutazione attiva.

Metodi e funzioni

SaveAssetCommand non definisce metodi.

5.3.2.3) SaveAssetService

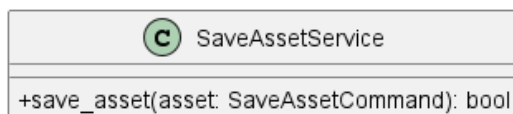


Figura 46: *SaveAssetService*

Descrizione

SaveAssetService è il service applicativo appartenente all'Application Core responsabile della logica di aggiornamento di un Asset esistente. Implementa l'interfaccia *SaveAssetUseCase*, recupera la sessione attiva tramite *GetSessionPort*, aggiorna l'Asset corrispondente e persiste la sessione modificata tramite *SaveSessionPort*.

Attributi

SaveAssetService non definisce attributi propri.

Metodi e funzioni

- *+ save_asset(asset: SaveAssetCommand): bool* — concretizza il contratto definito da *SaveAssetUseCase*. Recupera la sessione attiva, individua l'Asset da aggiornare tramite *asset_id* e ne persiste lo stato modificato.

5.3.3) DeleteAsset

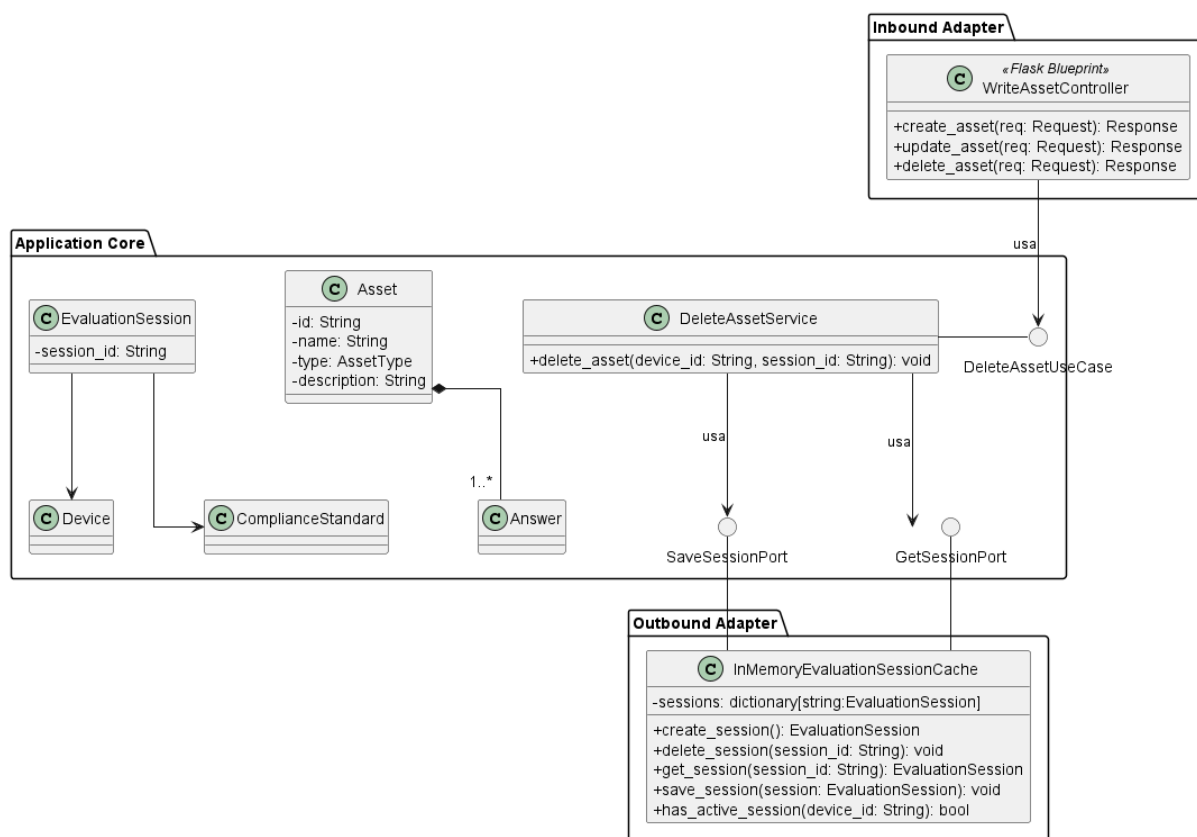


Figura 47: Caso d'uso DeleteAsset

Il diagramma illustra l'architettura del modulo di eliminazione di un Asset esistente all'interno di una sessione di valutazione attiva.

Per la definizione di *WriteAssetController*, vedere la sezione **Sezione 5.3.1.1**.

Per la definizione di *Asset*, vedere la sezione **Sezione 5.3.1.5**.

Per la definizione di *EvaluationSession*, vedere la sezione **Sezione 5.3.1.6**.

Per la definizione di *SaveSessionPort*, vedere la sezione **Sezione 5.3.1.8**.

Per la definizione di *GetSessionPort*, vedere la sezione **Sezione 5.3.1.9**.

Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.3.3.1) DeleteAssetUseCase



Figura 48: DeleteAssetUseCase

Descrizione

DeleteAssetUseCase è l'interfaccia (Inbound Port) che definisce il contratto per l'eliminazione di un Asset dalla sessione di valutazione. Viene implementata da *DeleteAssetService* e utilizzata da *WriteAssetController*.

Attributi

DeleteAssetUseCase non definisce attributi.

Metodi e funzioni

- `+ delete_asset(device_id: String, session_id: String): void` — firma del metodo delegato all'esecuzione della logica di eliminazione a partire dall'identificativo dell'Asset e della sessione.

5.3.3.2) DeleteAssetService

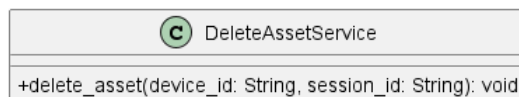


Figura 49: DeleteAssetService

Descrizione

DeleteAssetService è il service applicativo appartenente all'Application Core responsabile della logica di eliminazione di un Asset. Implementa l'interfaccia *DeleteAssetUseCase* e, a differenza dei service di creazione e modifica, non richiede un Command Object: riceve direttamente gli identificativi necessari, recupera la sessione tramite *GetSessionPort*, rimuove l'Asset corrispondente e persiste la sessione aggiornata tramite *SaveSessionPort*.

Attributi

DeleteAssetService non definisce attributi propri.

Metodi e funzioni

- `+ delete_asset(device_id: String, session_id: String): void` — concretizza il contratto definito da *DeleteAssetUseCase*. Recupera la sessione attiva, individua e rimuove l'Asset corrispondente e ne persiste lo stato aggiornato.

5.3.4) GetAssetDetail

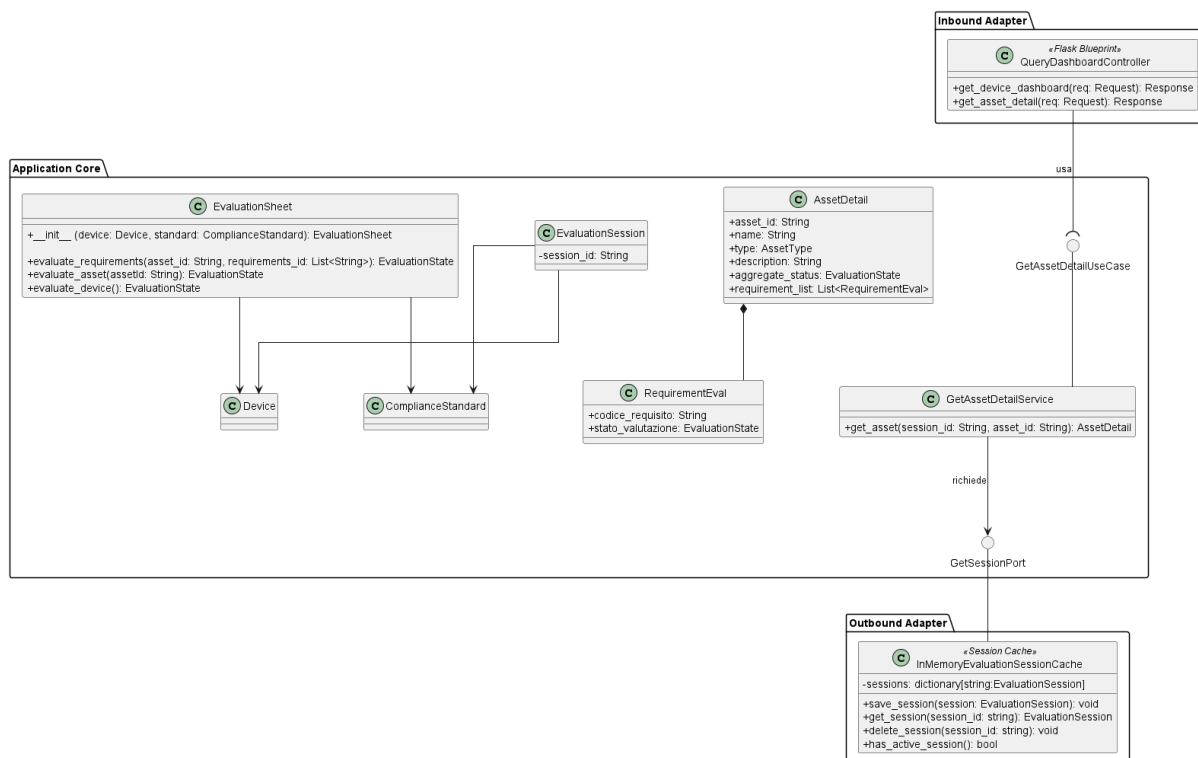


Figura 50: Diagramma delle classi — Caso d'uso GetAssetDetail

Il diagramma illustra l'architettura del modulo dedicato al recupero del dettaglio di un Asset all'interno di una sessione di valutazione attiva.

Per la definizione di *EvaluationSession*, vedere la sezione **Sezione 5.3.1.6**.

Per la definizione di *GetSessionPort*, vedere la sezione **Sezione 5.3.1.9**.

Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.3.4.1) QueryDashboardController



Figura 51: QueryDashboardController

Descrizione

QueryDashboardController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP di lettura relative alla dashboard e agli Asset. Delega la logica applicativa al livello sottostante tramite le rispettive interfacce.

Attributi

QueryDashboardController non definisce attributi propri.

Metodi e funzioni

- `+ get_device_dashboard(req: Request): Response` — riceve la richiesta HTTP di recupero della dashboard del Dispositivo e restituisce una risposta HTTP con i dati aggregati.
- `+ get_asset_detail(req: Request): Response` — riceve la richiesta HTTP di recupero del dettaglio di un Asset specifico e restituisce una risposta HTTP con i dati completi.

5.3.4.2) GetAssetDetailUseCase

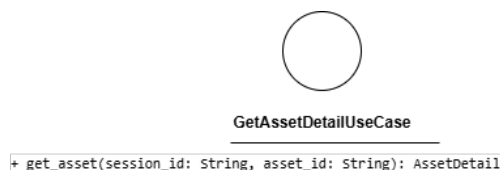


Figura 52: GetAssetDetailUseCase

Descrizione

GetAssetDetailUseCase è l'interfaccia (Inbound Port) che definisce il contratto per il recupero delle informazioni dettagliate relative a un singolo Asset. Viene utilizzata per visualizzare i parametri specifici di una risorsa (come sensori o attuatori) all'interno delle dashboard di monitoraggio.

Attributi

GetAssetDetailUseCase non definisce attributi.

Metodi e funzioni

- `+ get_detail(device_id: String, asset_id: String): AssetDetail` — firma del metodo che, dati gli identificativi del Dispositivo e dell'Asset, restituisce un oggetto *AssetDetail* contenente tutte le informazioni informative e lo stato corrente della risorsa.

Descrizione

GetAssetDetailUseCase è l'interfaccia (Inbound Port) che definisce il contratto per il recupero del dettaglio di un Asset. Viene implementata da *GetAssetDetailService* e utilizzata da *QueryDashboardController*.

Attributi

GetAssetDetailUseCase non definisce attributi.

Metodi e funzioni

- `+ get_asset(session_id: String, asset_id: String): AssetDetail` — firma del metodo delegato al recupero del dettaglio dell'Asset corrispondente agli identificativi forniti.

5.3.4.3) GetAssetDetailService

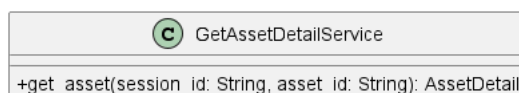


Figura 53: GetAssetDetailService

Descrizione

GetAssetDetailService è il service applicativo appartenente all'Application Core responsabile del recupero del dettaglio di un Asset. Implementa l'interfaccia *GetAssetDetailUseCase*, recupera la sessione attiva tramite *GetSessionPort* e costruisce il *AssetDetail* aggregando le informazioni dell'Asset con il suo stato di valutazione.

Attributi

GetAssetDetailService non definisce attributi propri.

Metodi e funzioni

- + `get_asset(session_id: String, asset_id: String): AssetDetail` — concretizza il contratto definito da *GetAssetDetailUseCase*. Recupera la sessione attiva, individua l'Asset richiesto e ne costruisce la rappresentazione *AssetDetail*.

5.3.4.4) AssetDetail

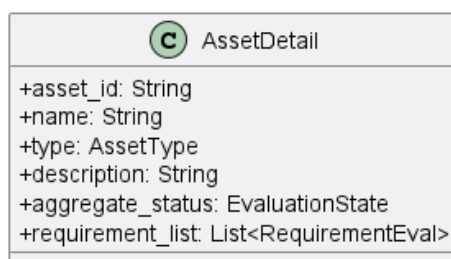


Figura 54: AssetDetail

Descrizione

AssetDetail è il Data Transfer Object che veicola la rappresentazione completa di un Asset verso il livello di presentazione. Aggrega le informazioni dell'Asset con il suo stato di valutazione e la lista dei requisiti valutati, evitando di esporre direttamente l'entità di dominio.

Attributi

- + `asset_id: String` — identificativo univoco dell'Asset.
- + `name: String` — nome dell'Asset.
- + `type: AssetType` — tipo dell'Asset.
- + `description: String` — descrizione testuale dell'Asset.
- + `aggregate_status: EvaluationState` — stato di valutazione aggregato dell'Asset.
- + `requirement_list: List<RequirementEval>` — lista dei requisiti valutati associati all'Asset.

Metodi e funzioni

AssetDetail non definisce metodi.

5.3.4.5) RequirementEval

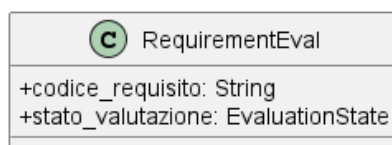


Figura 55: RequirementEval

Descrizione

RequirementEval è il Data Transfer Object che rappresenta la valutazione di un singolo requisito di conformità associato all'Asset. È contenuto nella lista *requirement_list* di *AssetDetail*.

Attributi

- + `codice_requisito`: String — codice identificativo del requisito di conformità.
- + `stato_valutazione`: EvaluationState — stato di valutazione del requisito.

Metodi e funzioni

RequirementEval non definisce metodi.

5.3.4.6) EvaluationSheet

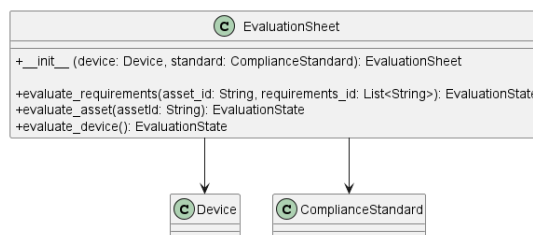


Figura 56: EvaluationSheet

Descrizione

EvaluationSheet è l'entità di dominio che coordina la valutazione di conformità di un Asset rispetto a uno standard. Espone i metodi necessari per valutare i requisiti e aggregare lo stato complessivo della valutazione.

Attributi

EvaluationSheet non definisce attributi propri nel diagramma.

Metodi e funzioni

- + `__init__(device: Device, standard: ComplianceStandard): EvaluationSheet` — inizializza il foglio di valutazione per il Dispositivo e lo standard forniti.
- + `evaluate_requirements(asset_id: String, requirements_id: List<String>): EvaluationState` — valuta i requisiti specificati per l'Asset indicato e restituisce lo stato risultante.

- + `evaluate_asset(asset: Asset): EvaluationState` — valuta lo stato complessivo dell'Asset fornito.
- + `evaluate_device(): EvaluationState` — valuta lo stato complessivo del Dispositivo associato alla sessione.

5.4) Session

5.4.1) OpenEvaluationSession

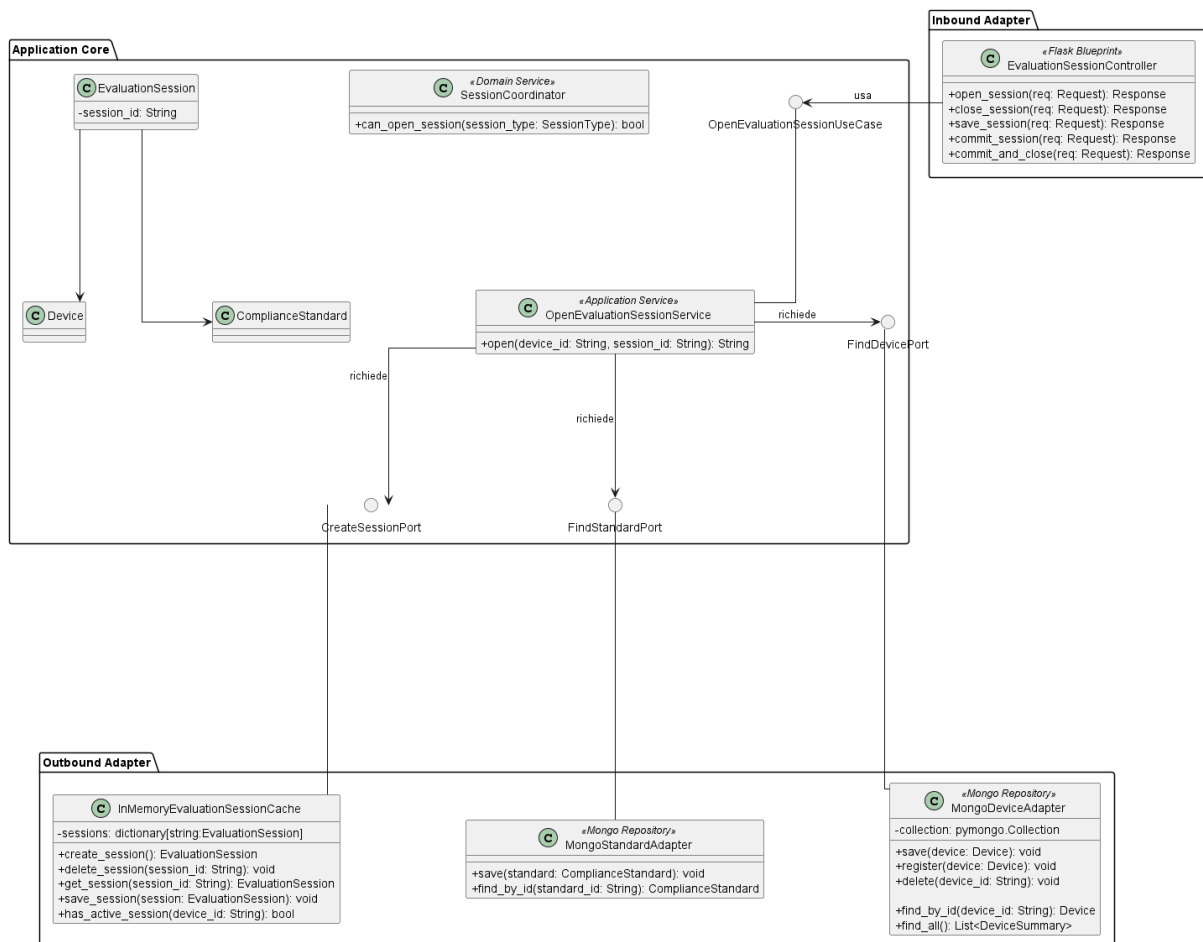


Figura 57: Caso d'uso OpenEvaluationSession

Il diagramma illustra l'architettura del modulo dedicato all'apertura di una sessione di valutazione.

- Per la definizione di *EvaluationSession*, vedere la sezione **Sezione 5.3.1.6**.
- Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7**.
- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.2.7**.
- Per la definizione di *FindDevicePort*, vedere la sezione **Sezione 5.2.6.4**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.4.1.0.1) EvaluationSessionController

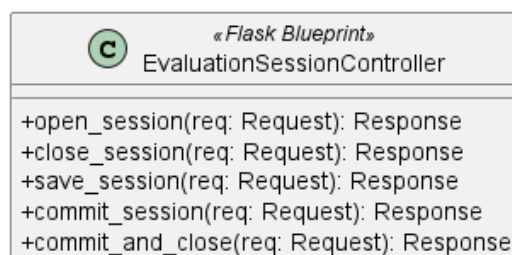


Figura 58: EvaluationSessionController

Descrizione

EvaluationSessionController è il controller Flask appartenente all'Inbound Adapter che riceve le richieste HTTP relative alla gestione del ciclo di vita della sessione di valutazione e le inoltra al livello applicativo.

Attributi

EvaluationSessionController non definisce attributi propri.

Metodi e funzioni

- `+ open_session(req: Request): Response` — riceve la richiesta HTTP di apertura di una nuova sessione di valutazione e restituisce una risposta HTTP con l'identificativo della sessione creata.
- `+ close_session(req: Request): Response` — riceve la richiesta HTTP di chiusura della sessione corrente e restituisce una risposta HTTP con l'esito dell'operazione.
- `+ save_session(req: Request): Response` — riceve la richiesta HTTP di salvataggio dello stato corrente della sessione e restituisce una risposta HTTP con l'esito dell'operazione.
- `+ commit_session(req: Request): Response` — riceve la richiesta HTTP di commit della sessione e restituisce una risposta HTTP con l'esito dell'operazione.
- `+ commit_and_close(req: Request): Response` — riceve la richiesta HTTP di commit e chiusura contestuale della sessione e restituisce una risposta HTTP con l'esito dell'operazione.

5.4.1.0.2) OpenEvaluationSessionUseCase



Figura 59: OpenEvaluationSessionUseCase

Descrizione

OpenEvaluationSessionUseCase è l'interfaccia (Inbound Port) che definisce il contratto per l'apertura di una nuova sessione di valutazione. Viene implementata da *OpenEvaluationSessionService* e utilizzata da *EvaluationSessionController*.

Attributi

OpenEvaluationSessionUseCase non definisce attributi.

Metodi e funzioni

- + open(device_id: String, session_id: String): String — firma del metodo delegato all'esecuzione della logica di apertura della sessione a partire dall'identificativo del Dispositivo e della sessione.

5.4.1.0.3) OpenEvaluationSessionService

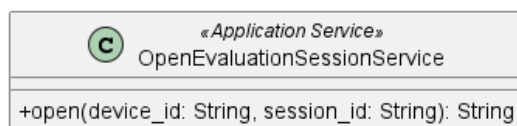


Figura 60: OpenEvaluationSessionService

Descrizione

OpenEvaluationSessionService è il service applicativo appartenente all'Application Core responsabile della logica di apertura di una sessione di valutazione. Implementa l'interfaccia *OpenEvaluationSessionUseCase* e coordina il recupero del Dispositivo tramite *FindDevicePort*, il recupero dello standard di conformità tramite *FindStandardPort* e la creazione della sessione tramite *CreateSessionPort*.

Attributi

OpenEvaluationSessionService non definisce attributi propri.

Metodi e funzioni

- + open(device_id: String, session_id: String): String — concretizza il contratto definito da *OpenEvaluationSessionUseCase*. Recupera il Dispositivo e lo standard associato, inizializza una nuova *EvaluationSession* e ne richiede la creazione tramite *CreateSessionPort*; restituisce l'identificativo della sessione creata.

5.4.1.0.4) SessionCoordinator

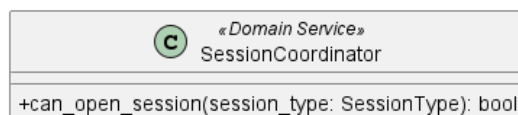


Figura 61: SaveAssetService

Descrizione

SessionCoordinator è il Domain Service che coordina la logica di dominio relativa alla gestione della sessione di valutazione. Verifica le precondizioni necessarie all'apertura di una sessione, come l'assenza di sessioni attive per il Dispositivo.

Attributi

SessionCoordinator non definisce attributi propri.

Metodi e funzioni

- `+ can_open_session(session_type: SessionType): bool` — verifica se è possibile aprire una nuova sessione del tipo specificato, restituendo `true` se le precondizioni sono soddisfatte.

5.4.1.0.5) CreateSessionPort

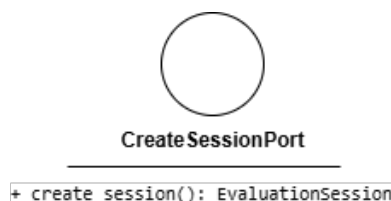


Figura 62: CreateSessionPort

Descrizione

CreateSessionPort è l'interfaccia (Outbound Port) che definisce il contratto per la creazione di una nuova sessione di valutazione nel sistema di persistenza in memoria. Viene implementata da *InMemoryEvaluationSessionCache* e utilizzata da *OpenEvaluationSessionService*.

Attributi

CreateSessionPort non definisce attributi.

Metodi e funzioni

- `+ create_session(): EvaluationSession` — firma del metodo che inizializza e registra una nuova sessione di valutazione nel sistema in memoria.

5.4.1.0.6) FindStandardPort

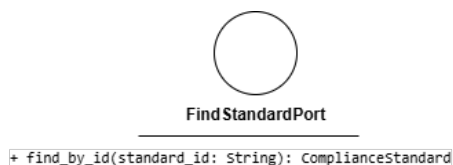


Figura 63: FindStandardPort

Descrizione

FindStandardPort è l'interfaccia (Outbound Port) che definisce il contratto per il recupero di uno standard di conformità dal sistema di persistenza. Viene implementata da *MongoStandardAdapter* e utilizzata da *OpenEvaluationSessionService*.

Attributi

FindStandardPort non definisce attributi.

Metodi e funzioni

- `+ find_by_id(standard_id: String): ComplianceStandard` — firma del metodo che recupera lo standard di conformità corrispondente all'identificativo fornito.

5.4.1.0.7) MongoStandardAdapter



Figura 64: MongoStandardAdapter

Descrizione

MongoStandardAdapter è la classe dell'Outbound Adapter annotata come *Mongo Repository* che implementa *FindStandardPort*, traducendo le operazioni di recupero degli standard di conformità in interazioni concrete con MongoDB.

Attributi

MongoStandardAdapter non definisce attributi propri nel diagramma.

Metodi e funzioni

- `+ save(standard: ComplianceStandard): void` — persiste uno standard di conformità nel database.
- `+ find_by_id(standard_id: String): ComplianceStandard` — recupera lo standard di conformità corrispondente all'identificativo fornito.

5.4.1.1) SaveEvaluationSession

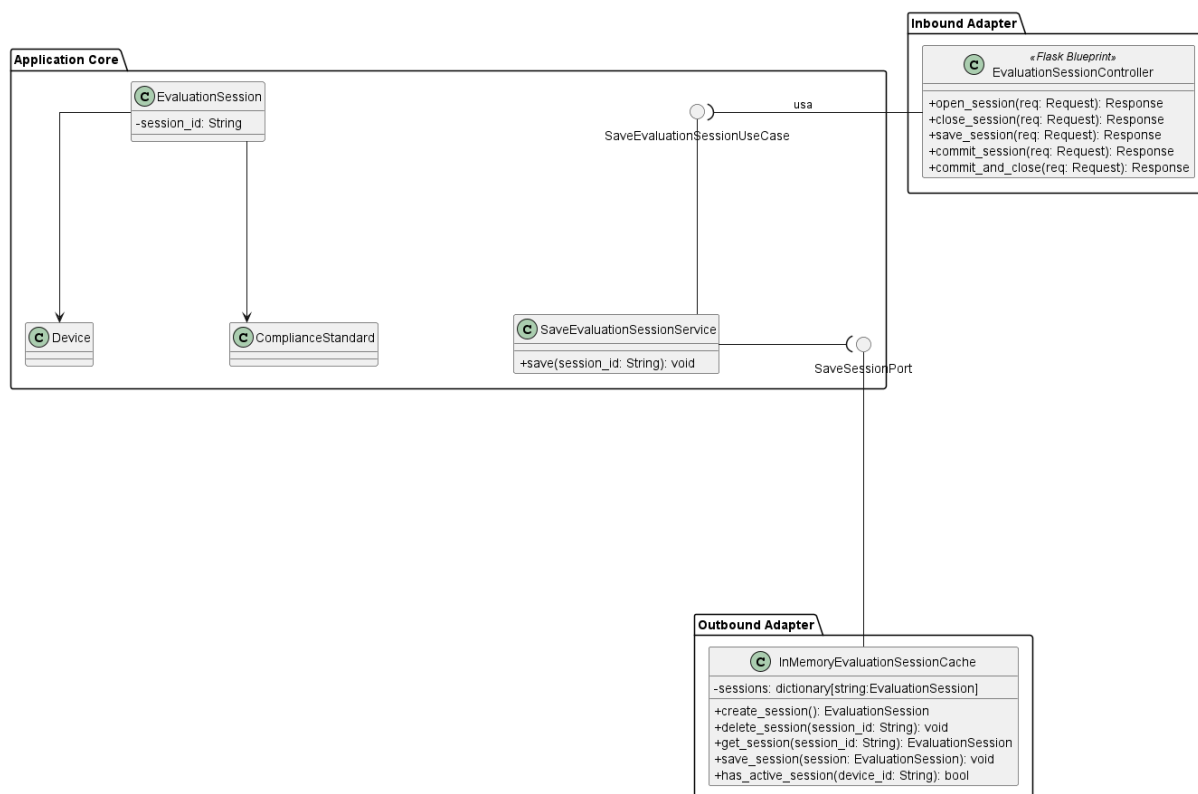


Figura 65: Caso d'uso SaveEvaluationSession

Il diagramma illustra l'architettura del modulo dedicato al salvataggio dello stato di una sessione di valutazione corrente.

- Per la definizione di *EvaluationSessionController*, vedere la sezione **Sezione 5.4.1.0.1**.
- Per la definizione di *EvaluationSession*, vedere la sezione **Sezione 5.3.1.6**.
- Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.4.1.1.1) SaveEvaluationSessionUseCase



Figura 66: SaveEvaluationSessionUseCase

Descrizione

SaveEvaluationSessionUseCase è l'interfaccia (Inbound Port) che definisce il contratto per il salvataggio dello stato di una sessione di valutazione attiva. Viene implementata da *SaveEvaluationSessionService* e utilizzata da *EvaluationSessionController*.

Attributi

SaveEvaluationSessionUseCase non definisce attributi.

Metodi e funzioni

- + save(session_id: String): void — firma del metodo delegato all'esecuzione della logica di salvataggio della sessione a partire dal suo identificativo.

5.4.1.1.2) SaveEvaluationSessionService

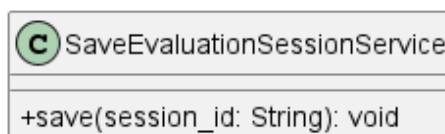


Figura 67: SaveEvaluationSessionService

Descrizione

SaveEvaluationSessionService è il service applicativo appartenente all'Application Core responsabile del coordinamento dell'operazione di salvataggio. Implementa l'interfaccia *SaveEvaluationSessionUseCase*. Recupera la sessione corrente e ne richiede la persistenza aggiornata tramite la porta di uscita *SaveSessionPort*.

Attributi

SaveEvaluationSessionService non definisce attributi propri.

Metodi e funzioni

- + save(session_id: String): void — concretizza il contratto definito da *SaveEvaluationSessionUseCase*. Coordina il salvataggio dello stato attuale della sessione di valutazione.

5.4.1.1.3) *SaveSessionPort*



Figura 68: *SaveSessionPort*

SaveSessionPort è l'interfaccia (Outbound Port) che definisce il contratto per la persistenza di una sessione di valutazione nel sistema di archiviazione (in questo caso, la cache in memoria). Viene implementata da *InMemoryEvaluationSessionCache* e utilizzata da *SaveEvaluationSessionService*.

Attributi

SaveSessionPort non definisce attributi.

Metodi e funzioni

- + `save_session(session: EvaluationSession): void` — firma del metodo che sovrascrive o aggiorna lo stato di una sessione di valutazione nel sistema di persistenza.

5.4.1.2) CloseEvaluationSession

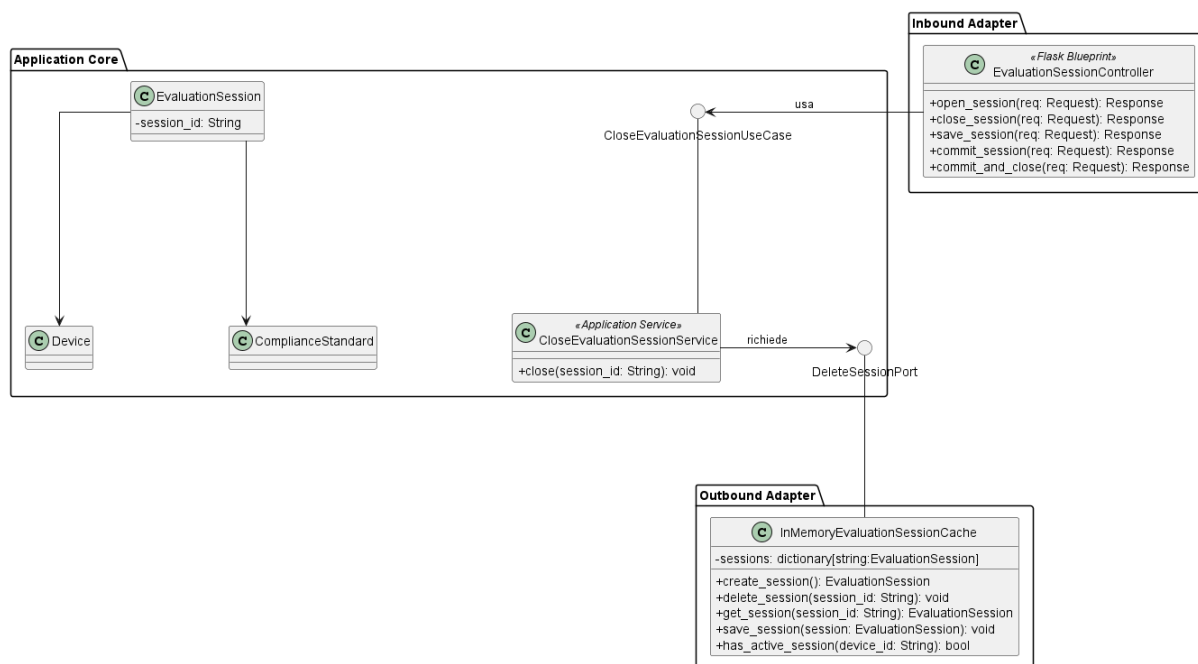


Figura 69: Caso d'uso CloseEvaluationSession

Il diagramma illustra l'architettura del modulo dedicato alla chiusura e all'eliminazione di una sessione di valutazione.

- Per la definizione di *EvaluationSessionController*, vedere la sezione **Sezione 5.4.1.0.1**.
- Per la definizione di *EvaluationSession*, vedere la sezione **Sezione 5.3.1.6**.
- Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7**.

Di seguito vengono documentati esclusivamente i componenti introdotti specificamente per questo caso d'uso.

5.4.1.2.1) CloseEvaluationSessionUseCase



Figura 70: CloseEvaluationSessionUseCase

Descrizione

CloseEvaluationSessionUseCase è l'interfaccia (Inbound Port) che definisce il contratto per la chiusura definitiva di una sessione di valutazione. Rappresenta l'operazione conclusiva del ciclo di vita della sessione, in cui i dati vengono consolidati e lo stato viene impostato come finalizzato.

Attributi

CloseEvaluationSessionUseCase non definisce attributi.

Metodi e funzioni

- `+ close(session_id: String): void` — firma del metodo che riceve l'identificativo della sessione da terminare. L'implementazione si occupa di marcare la sessione come chiusa e di innescare eventuali logiche di post-elaborazione o archiviazione.

Descrizione

CloseEvaluationSessionUseCase è l'interfaccia (Inbound Port) che definisce il contratto per la chiusura di una sessione di valutazione attiva. Viene implementata da *CloseEvaluationSessionService* e utilizzata dal controller *EvaluationSessionController*.

Attributi

CloseEvaluationSessionUseCase non definisce attributi.

Metodi e funzioni

- `+ close(session_id: String): void` — firma del metodo delegato all'esecuzione della logica di chiusura della sessione a partire dal suo identificativo.

5.4.1.2.2) CloseEvaluationSessionService

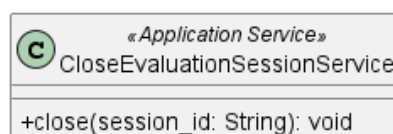


Figura 71: CloseEvaluationSessionService

Descrizione

CloseEvaluationSessionService è il service applicativo appartenente all'Application Core responsabile della logica di chiusura della sessione. Implementa l'interfaccia *CloseEvaluationSessionUseCase*. Una volta eseguite le operazioni di dominio necessarie, richiede la rimozione della sessione dal sistema di persistenza richiamando la porta in uscita *DeleteSessionPort*.

Attributi

CloseEvaluationSessionService non definisce attributi propri.

Metodi e funzioni

- + `close(session_id: String): void` — concretizza il contratto definito da *CloseEvaluationSessionUseCase*. Coordina le operazioni di chiusura e inoltra la richiesta di eliminazione della sessione specificata.

5.4.1.2.3) DeleteSessionPort



Figura 72: DeleteSessionPort

Descrizione

DeleteSessionPort è l'interfaccia (Outbound Port) che definisce il contratto per l'eliminazione di una sessione di valutazione dal sistema di archiviazione (es. la cache in memoria). Viene utilizzata da *CloseEvaluationSessionService* e implementata nel livello di adapter da *InMemoryEvaluationSessionCache*.

Attributi

DeleteSessionPort non definisce attributi.

Metodi e funzioni

- + `delete_session(session_id: String): void` — firma del metodo che si occupa di rimuovere o invalidare lo stato di una specifica sessione di valutazione dal sistema di persistenza.

5.4.1.3) CommitEvaluationSession

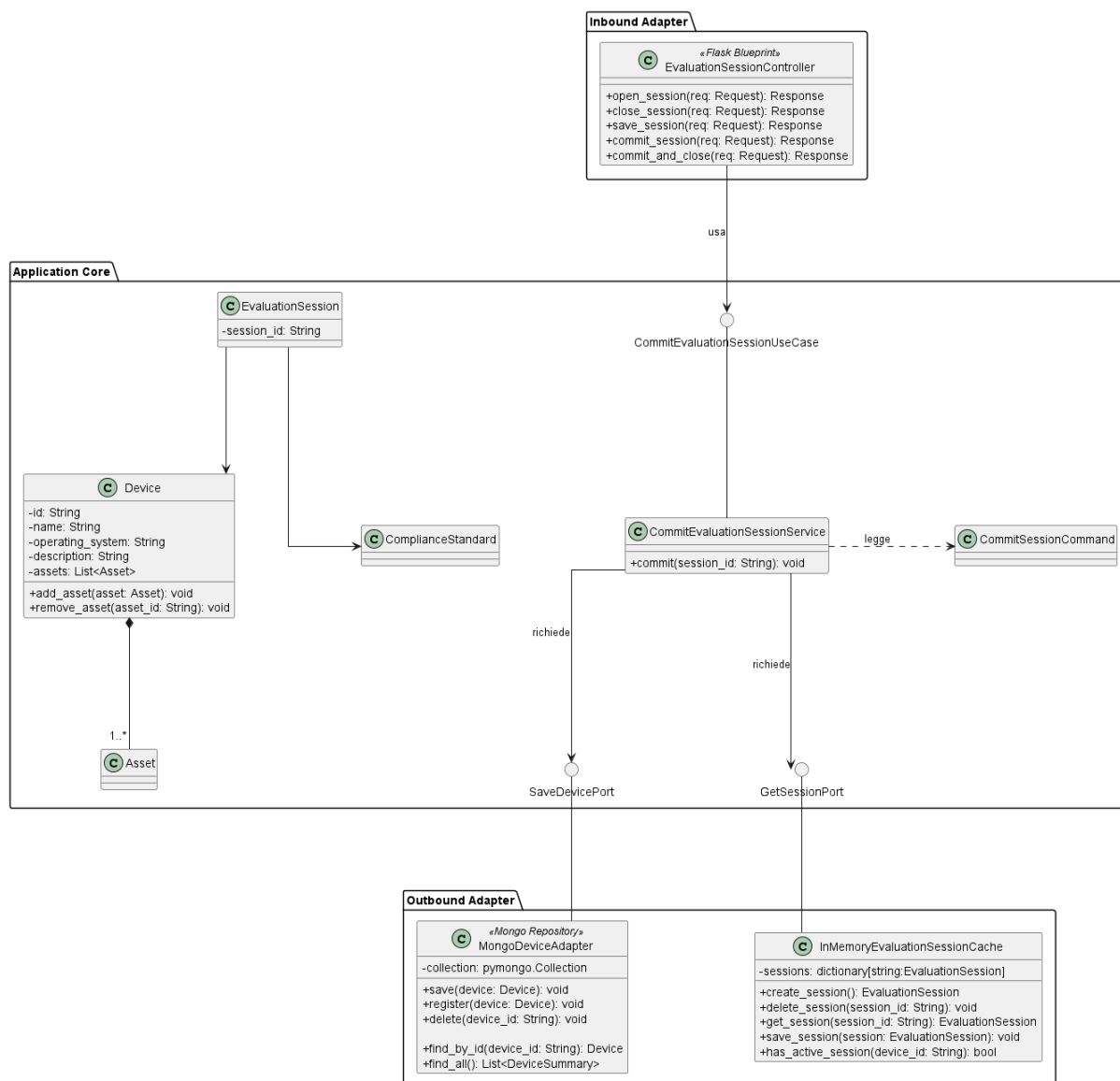


Figura 73: Caso d'uso CommitEvaluationSession

Il diagramma illustra l'architettura del modulo dedicato esclusivamente al consolidamento (commit) dei dati di una sessione di valutazione verso il dispositivo, senza richiederne la chiusura o l'eliminazione dalla memoria temporanea.

- Per la definizione di *EvaluationSessionController*, vedere la sezione **Sezione 5.4.1.0.1**.
- Per la definizione di *EvaluationSession*, vedere la sezione **Sezione 5.3.1.6**.
- Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7**.
- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.2.7**.
- Per la definizione di *SaveDevicePort*, vedere la sezione **Sezione 5.2.4.4**.
- Per la definizione di *GetSessionPort*, vedere la sezione **Sezione 5.3.1.9**.

Di seguito vengono documentati esclusivamente i componenti specifici introdotti per questo flusso operativo.

5.4.1.3.1) CommitEvaluationSessionService

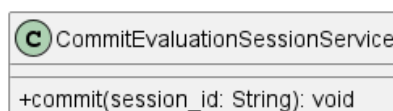


Figura 74: CommitEvaluationSessionService

CommitEvaluationSessionService è il service applicativo dell'Application Core responsabile di orchestrare l'operazione di commit. Implementa l'interfaccia *CommitEvaluationSessionUseCase*. Coordina il recupero della sessione attualmente in corso (tramite la porta *GetSessionPort*), legge i dati necessari dal *CommitSessionCommand* e applica le modifiche definitive delegando il salvataggio all'entità dispositivo (tramite *SaveDevicePort*).

Attributi

CommitEvaluationSessionService non definisce attributi propri.

Metodi e funzioni

- `+ commit(session_id: String): void` — concretizza la logica di business relativa al consolidamento dei dati. Utilizza l'identificativo della sessione per applicare le modifiche allo stato persistente del dispositivo.

5.4.1.3.2) CommitEvaluationSessionUseCase



Figura 75: CommitEvaluationSessionUseCase

Descrizione

CommitEvaluationSessionUseCase è l'interfaccia (Inbound Port) che definisce il contratto per eseguire il consolidamento (commit) dei dati di una sessione di valutazione attiva, applicando definitivamente le modifiche all'entità dispositivo associata. Viene implementata a livello applicativo e utilizzata dal controller *EvaluationSessionController*.

Attributi

CommitEvaluationSessionUseCase non definisce attributi.

Metodi e funzioni

- + `commit(session_id: String): void` — firma del metodo delegato all'esecuzione della logica di consolidamento dei dati della sessione di valutazione a partire dal suo identificativo univoco (`session_id`).

5.4.1.4) CommitCloseSession

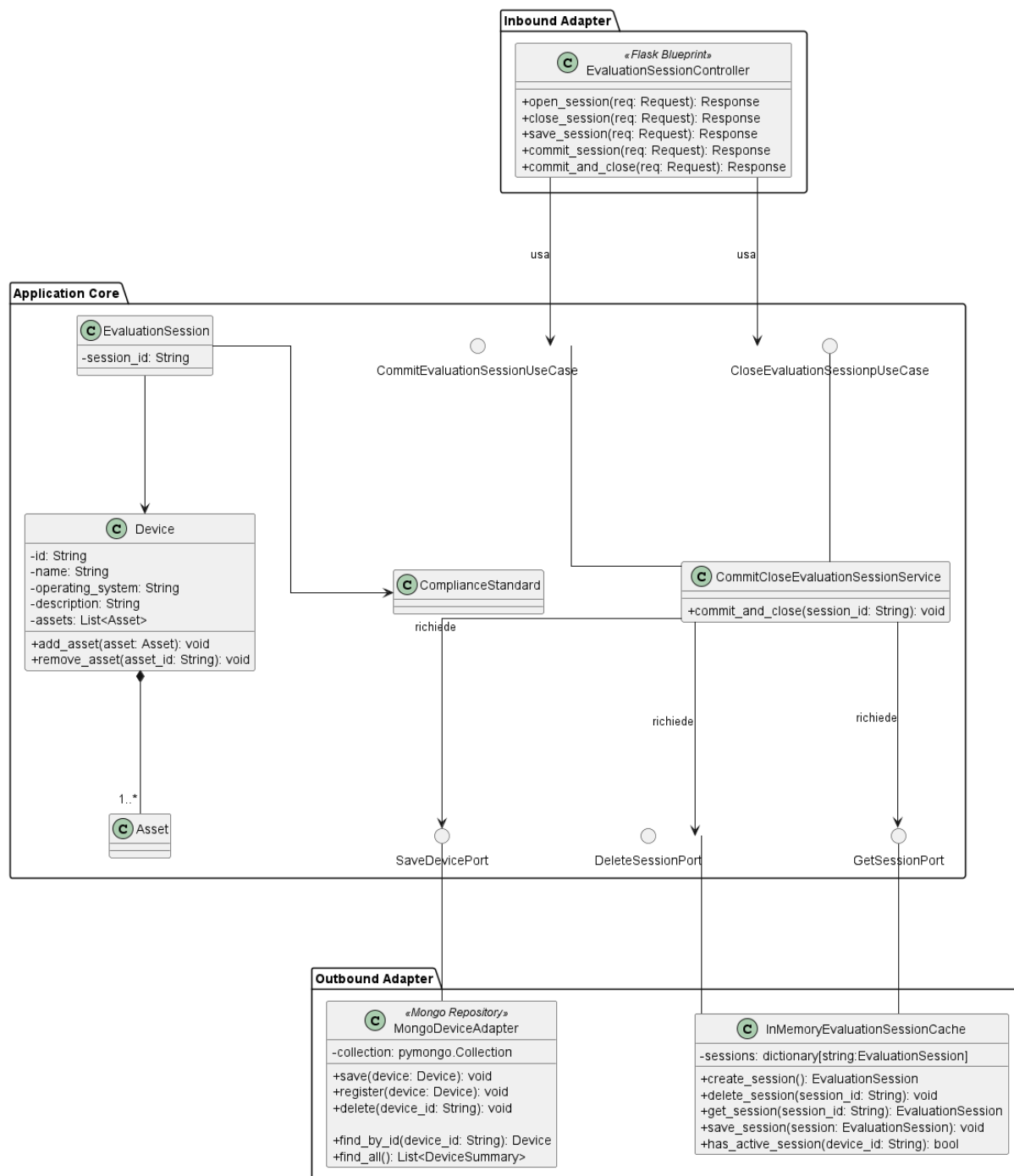


Figura 76: Caso d'uso CommitCloseSession

Il diagramma illustra l'architettura del modulo dedicato al consolidamento (commit) e alla contestuale chiusura di una sessione di valutazione.

- Per la definizione di *EvaluationSessionController*, vedere la sezione **Sezione 5.4.1.0.1**.

- Per la definizione di *EvaluationSession*, vedere la sezione **Sezione 5.3.1.6**.
- Per la definizione di *Device*, vedere la sezione **Sezione 5.2.2.5**.
- Per la definizione di *InMemoryEvaluationSessionCache*, vedere la sezione **Sezione 5.3.1.7**.
- Per la definizione di *MongoDeviceAdapter*, vedere la sezione **Sezione 5.2.2.7**.
- Per la definizione di *CloseEvaluationSessionUseCase*, vedere la sezione **Sezione 5.4.1.2.1**.
- Per la definizione di *DeleteSessionPort*, vedere la sezione **Sezione 5.4.1.2.3**.
- Per la definizione di *SaveDevicePort*, vedere la sezione **Sezione 5.2.4.4**.
- Per la definizione di *CommitEvaluationSessionUseCase*, vedere la sezione **Sezione 5.4.1.3.2**.

Di seguito vengono documentati esclusivamente i componenti introdotti o aggregati specificamente per questo flusso operativo.

5.4.1.4.1) CommitCloseEvaluationSessionService

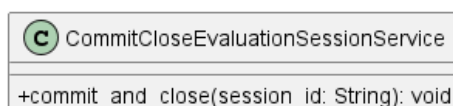


Figura 77: CommitCloseEvaluationSessionService

Descrizione

CommitCloseEvaluationSessionService è il service applicativo dell'Application Core responsabile di orchestrare le operazioni combinate. Fungendo da implementazione per i casi d'uso di commit e chiusura, coordina il recupero della sessione in corso (tramite *GetSessionPort*), il salvataggio dei risultati consolidati sul dispositivo (tramite *SaveDevicePort*) e, in caso di successo, la rimozione della sessione attiva (tramite *DeleteSessionPort*).

Attributi

CommitCloseEvaluationSessionService non definisce attributi propri.

Metodi e funzioni

- + `commit_and_close(session_id: String): void` — concretizza la logica di business principale. Prende in input l'identificativo della sessione, ne effettua il commit dei dati sull'entità *Device* e successivamente procede con l'eliminazione della sessione dalla memoria temporanea.

5.5) Importazione ed Esportazione Dispositivi

Per realizzare le funzioni di importazione ed esportazione dei dispositivi tramite file esterno rispettando i principi dell'architettura esagonale si è deciso di modellare il sistema mettendo in evidenza l'ambito di competenza delle varie classi.

5.5.1) Importazione Dispositivo

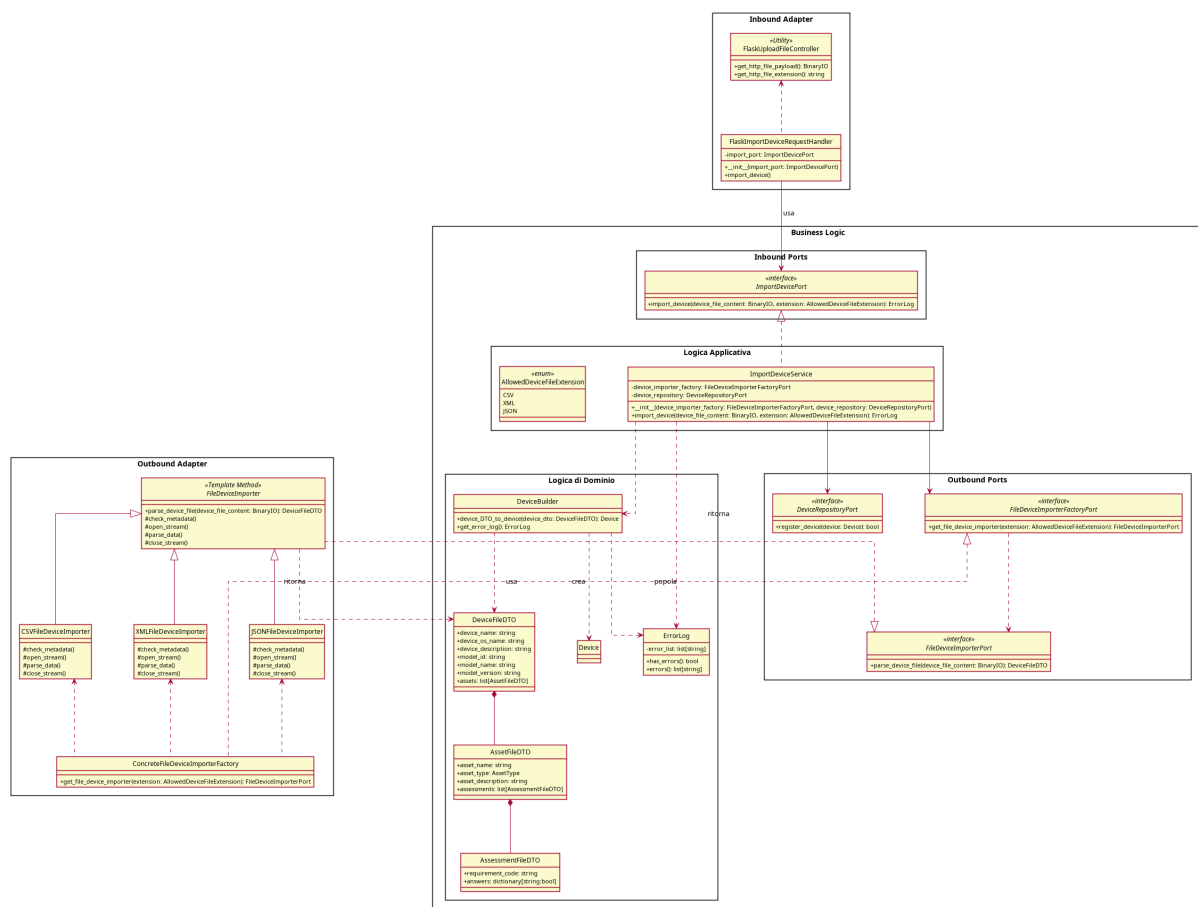


Figura 78: Diagramma delle classi - Importazione Dispositivo

Le classi dell'inbound adapter gestiscono la ricezione del file dati in ingresso usando le funzionalità di Flask.

L'Inbound Port è un'interfaccia funzionale che espone un metodo che accetta il file come parametro un oggetto di una classe della libreria Python standard

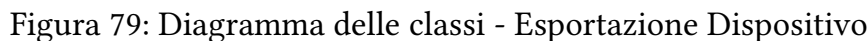
La porta è implementata da un Service che realizza la logica applicativa.

Il service si avvale di una classe di Dominio apposita: DeviceFileDTO, questa scelta è stata presa in quanto è stato ritenuto più appropriato l'uso di un oggetto privo di comportamento durante il processo di importazione ed esportazione

Per gestire la conversione da DTO a oggetti di dominio si usa una classe utility che converte il DTO in un oggetto di dominio che può essere inoltrato al sistema di permanenza

Vi sono altri servizi definiti esternamente tramite le outbound ports device file importer e device file importer factory, il factory serve a non dare al service la responsabilità di creazione dell'importer appropriato al formato di file caricato.

5.5.2) Esportazione dispositivo



L'Inbound Port è un'interfaccia funzionale che espone un metodo che accetta come parametri un id del dispositivo da esportare e come tipo di ritorno una classe della libreria standard

Il service si avvale di una classe di Dominio apposita: DeviceFileDTO, questa scelta è stata presa in quanto è stato ritenuto più appropriato l'uso di un oggetto privo di comportamento durante il processo di importazione ed esportazione

L'exporter è implementato tramite un template method le cui implementazioni concrete sono create dal factory

Il service si avvale di una classe di Dominio apposita: ModelFileDTO, questa scelta è stata presa in quanto è stato ritenuto più appropriato l'uso di un oggetto privo di comportamento durante il processo di importazione ed esportazione

Per gestire la conversione da DTO a oggetti di dominio si usa una classe utility che converte il DTO in un oggetto di dominio che può essere inoltrato al sistema di permanenza

Vi sono altri servizi definiti esternamente tramite le outbound ports model file importer e model file importer factory, il factory serve a non dare al service la responsabilità di creazione dell'importer appropriato al formato di file caricato.

L'importer è implementato tramite un template method le cui implementazioni concrete sono create dal factory

5.6.2) Esportazione modello

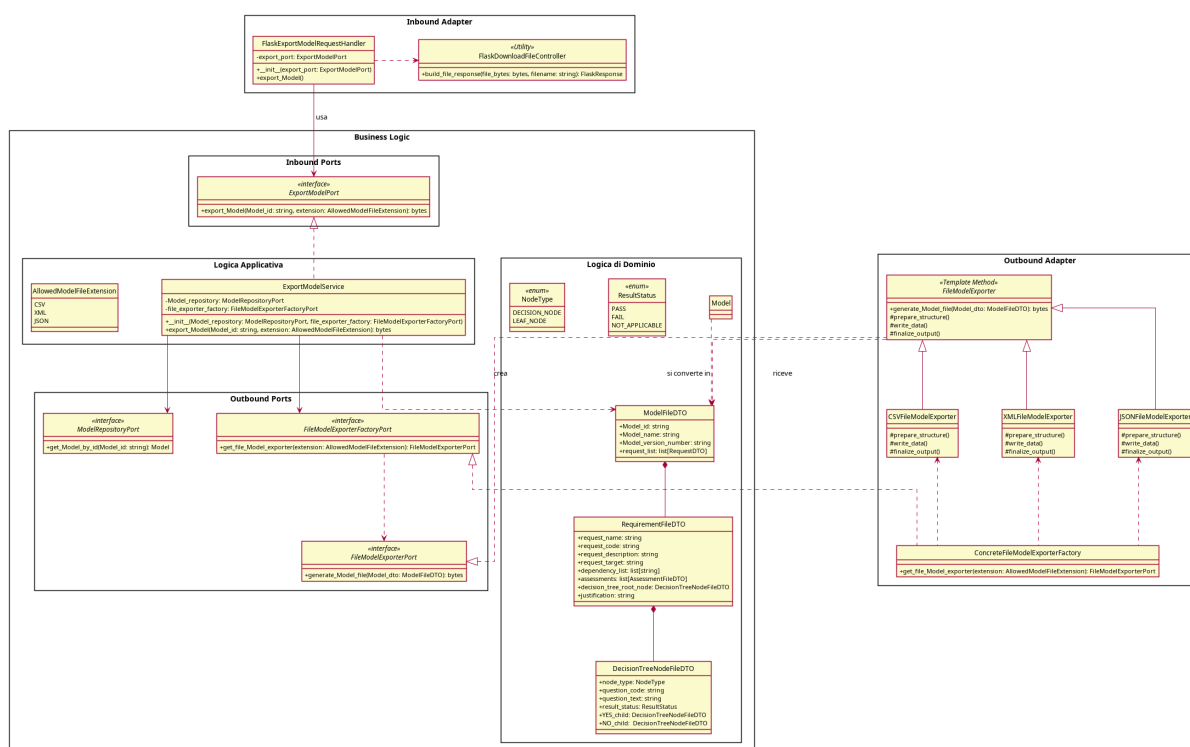


Figura 81: Diagramma delle classi - Esportazione Modello

Le classi dell'inbound adapter gestiscono l'invio del file dati in uscita usando le funzionalità di Flask.

L'Inbound Port è un'interfaccia funzionale che espone un metodo che accetta come parametri un id del modello da esportare e ritorna il file generato come una classe della libreria

La porta è implementata da un Service che realizza la logica applicativa.

Il service si avvale di una classe di Dominio apposita: ModelFileDTO, questa scelta è stata presa in quanto è stato ritenuto più appropriato l'uso di un oggetto privo di comportamento durante il processo di importazione ed esportazione

Vi sono servizi definiti esternamente tramite le outbound ports model file exporter e model file exporter factory, il factory serve a non dare al service la responsabilità di creazione dell'exporter appropriato al formato di file richiesto.

L'exporter è implementato tramite un template method le cui implementazioni concrete sono create dal factory

5.6.3) Classi Condivise

Essendo importazione ed esportazione 2 processi strettamente collegati, è stato ritenuto accettabile mantenere una classe condivisa di DTO, in quanto i dati necessari sono gli stessi sia durante il processo di importazione sia di esportazione

5.7) Generazione Report

Per realizzare la funzione di generazione del report di valutazione come file scaricabile rispettando i principi dell'architettura esagonale si è deciso di modellare il sistema mettendo in evidenza l'ambito di competenza delle varie classi.

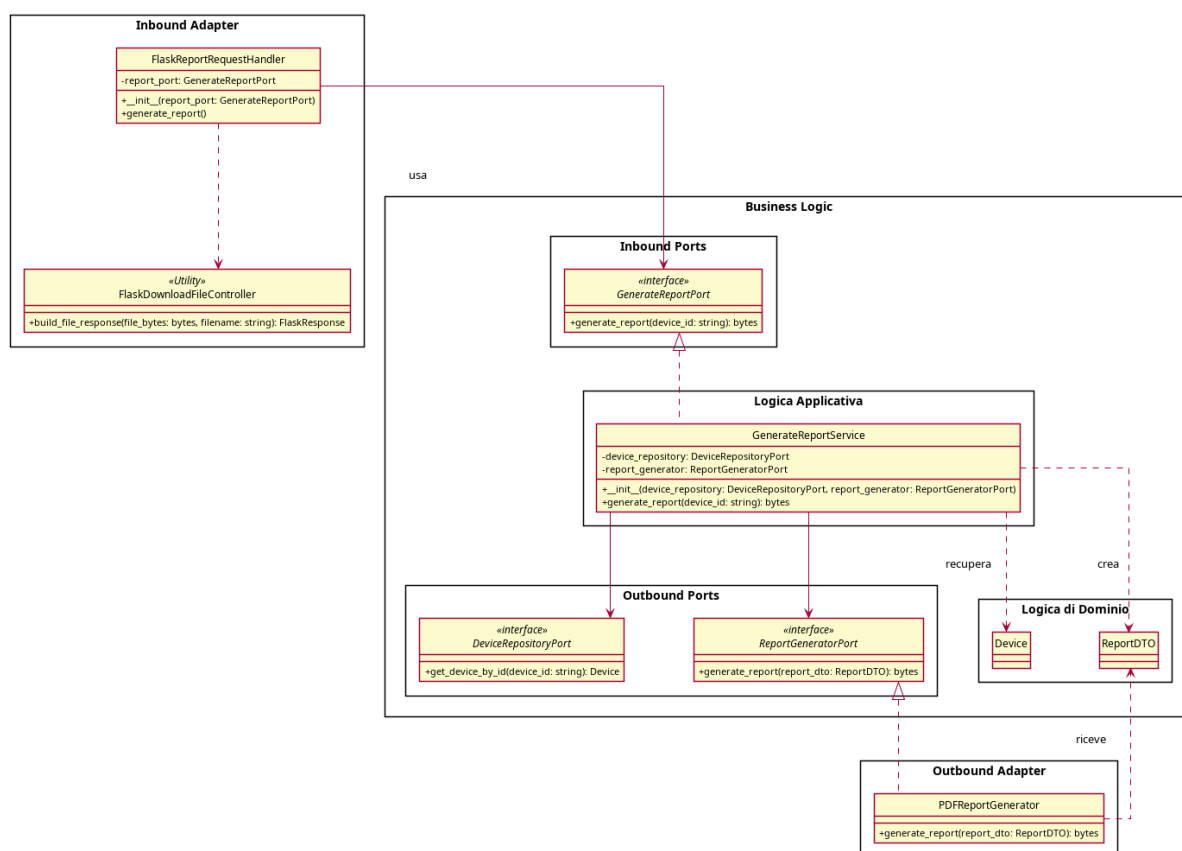


Figura 82: Diagramma delle classi - Importazione Modello

Le classi dell'inbound adapter gestiscono l'invio del file dati in uscita usando le funzionalità di Flask.

L'Inbound Port è un'interfaccia funzionale che espone un metodo che accetta come parametri un id del modello su cui eseguire la valutazione e ritorna il file di report generato come una classe della libreria standard

La porta è implementata da un Service che realizza la logica applicativa.

Il service si avvale di una classe di Dominio apposita: reportDTO, questa scelta è stata presa in quanto è stato ritenuto più appropriato l'uso di un oggetto privo di comportamento durante il processo di importazione ed esportazione

La generazione del file di report è gestita come servizio esterno con una porta dedicata e implementata da una classe esterna, non viene usato un factory in quanto è da supportare la generazione di una sola forma di report.

6) Tracciamento

7) Qualità architettuale

8) Gestione Errori e Logging

9) Sicurezza

10) Performance e Scalabilità